LECTURE 03

# 软件仓库管理

03

从一份目录到可解释、可恢复、可协作的项目历史。

25 节 · 53 幅配图 · 5 道自测　回看课堂 ↗

完整阅读本次 BaoCut original 导出的533句、20个分块，正文按25个语义段落组织；转写未进行整片人工听校，专业词依原讲义及画面核对，仍可能有细小听辨误差。

逐页目视官方35张讲义截图，另查看19个现场候选帧并选入18张；未逐秒观看视频。截图汇编 PDF 为本次存档，不是官方原生 PDF。

现行讲义与视频展开范围不同：SWE-bench/Astropy案例、Agent VCS等明确标为讲义补充。官方来源本次复用主协调保存的网页，未假装重新访问所有引文或验证当前产品能力。

桌面与窄屏浏览器验收由主协调记录；真实手机未测试。

阅读导引

## 先把握本课主线

01 Git 的核心是用有意义的**原子变更**推进**项目快照**，使历史可以恢复、理解、协作和审查。

02 学习方法是先在脑中实现，再对照真实工具；从工作区、Index、对象和引用的实际变化建立概念。

03 AI 能生成教学工具和大量实现，但架构、验收条件与组合后的**正确性**仍需要明确的判断。

第 01 节　00:00:00—00:02:39 · 课堂时间

# 模型进步以后，开放的设计任务仍需要反馈

> **关键判断 · 整理者提炼**
> 设计意图需要在实现与反馈中逐步清楚，模型进步并不取消验收判断。

前两讲讲如何把 AI 放进生活与工作的流程，清楚表达**真实意图**，再把重复流程沉淀为 skills。本课从这种个人使用经验转入软件项目：如何开始、保存、修改和协作。

讲师用 GPT-6 的使用感受开场：复杂任务更快，一些能力有惊喜，但**产品品位**仍需要反复反馈。这里是课堂时点的个人观察，没有统一评测、固定任务或完整性能数据，不应把它当成模型普遍排名。




1 / 35

**资料图 · 第 1 页** 课堂开场的模型进展话题；保留课堂语境，不作当前产品能力评测。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 1 · 原始讲义 · 课堂开场的模型进展话题；保留课堂语境，不作当前产品能力评测。

问题在于 Intent–Spec–Impl 的选择空间很开放。同一句“做一个国民级应用”，不同产品经理心里的目标也不同；模型只能用自己的理解填补未说清楚的部分。于是实现一出来，人才能看见自己真正介意的细节，需求又随之改变。

整理者说明

讲义补充：借 Simon 对结构良好/结构不良问题边界流动性的讨论，说明反馈可以把开放愿望逐步变成明确约束。




**资料图 · 第 2 页** 设计问题具有开放的意图、规约和实现空间。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 2 · 原始讲义 · 设计问题具有开放的意图、规约和实现空间。

讲师还感觉生成代码越来越紧凑、自己越来越不愿逐行阅读。这是其工作方式的描述；不能据此推导任意项目都可以跳过**正确性与验收判断**。

---

第 02 节

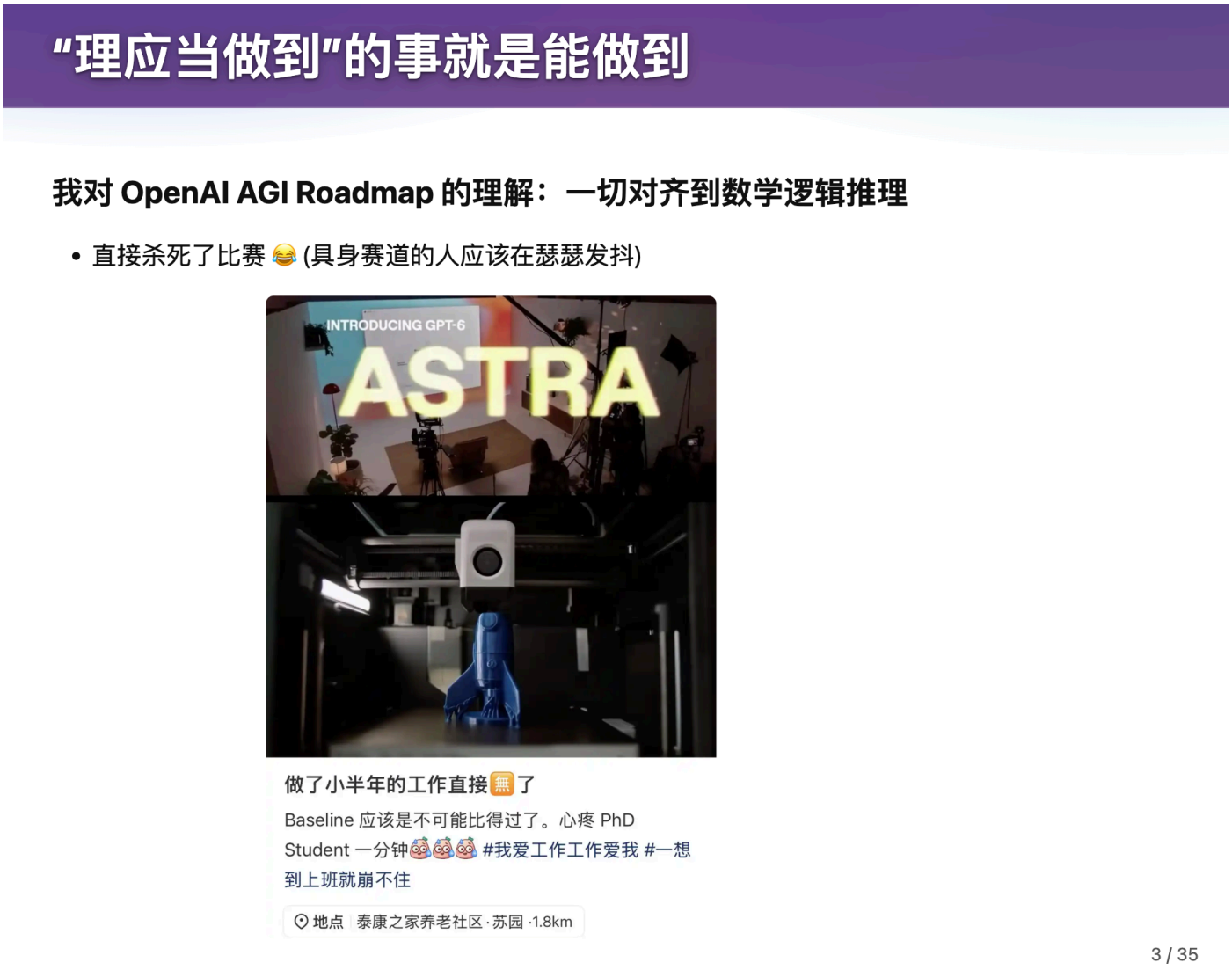
00:02:39—00:06:46 · 课堂时间

## 数学、空间理解与研究选题变化：保留推测的边界

**关键判断 · 整理者提炼**

数学与空间演示是讲师观察到的案例，不能据此推断普遍能力或内部训练路线。

讲师对模型发展路线的个人理解，是先把**数学逻辑**训练得可靠，再向其他能力扩展。数学的优势在于共享的推理规则和**可检验结论**，比“好用”“有品位”更容易形成共同标准。这里不是对 OpenAI 内部训练计划的确认。




**资料图 · 第 3 页** “理应当做到”与数学逻辑路线，是讲师在课堂上的判断和研究经历。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 3 · 原始讲义 · “理应当做到”与数学逻辑路线，是讲师在课堂上的判断和研究经历。

他用午饭选择作类比：在哪里吃、是否赶得及会议、能否午睡，都是多个约束与偏好的组合；日常行为中也包含可以表达的推理关系。空间题如立方体投影、骰子展开图、鲁班锁抽块，也可能从简单任务逐步训练。

研究案例是对话生成符合约束的 **3D 打印模型**。团队曾用较长搜索过程制作从左右两边看到不同图案的光栅玩具；讲师说新模型在该例中可一次生成，因而原先半年的研究工作面临重新定位。

他又提及网上机械臂抓取、叠衣服演示：逐帧给图、询问下一步动作，也能出现较强效果。课堂没有提供可重复的机器人实验设计或**成功率**，本笔记只保留它作为讲师感受到能力进步的例证。

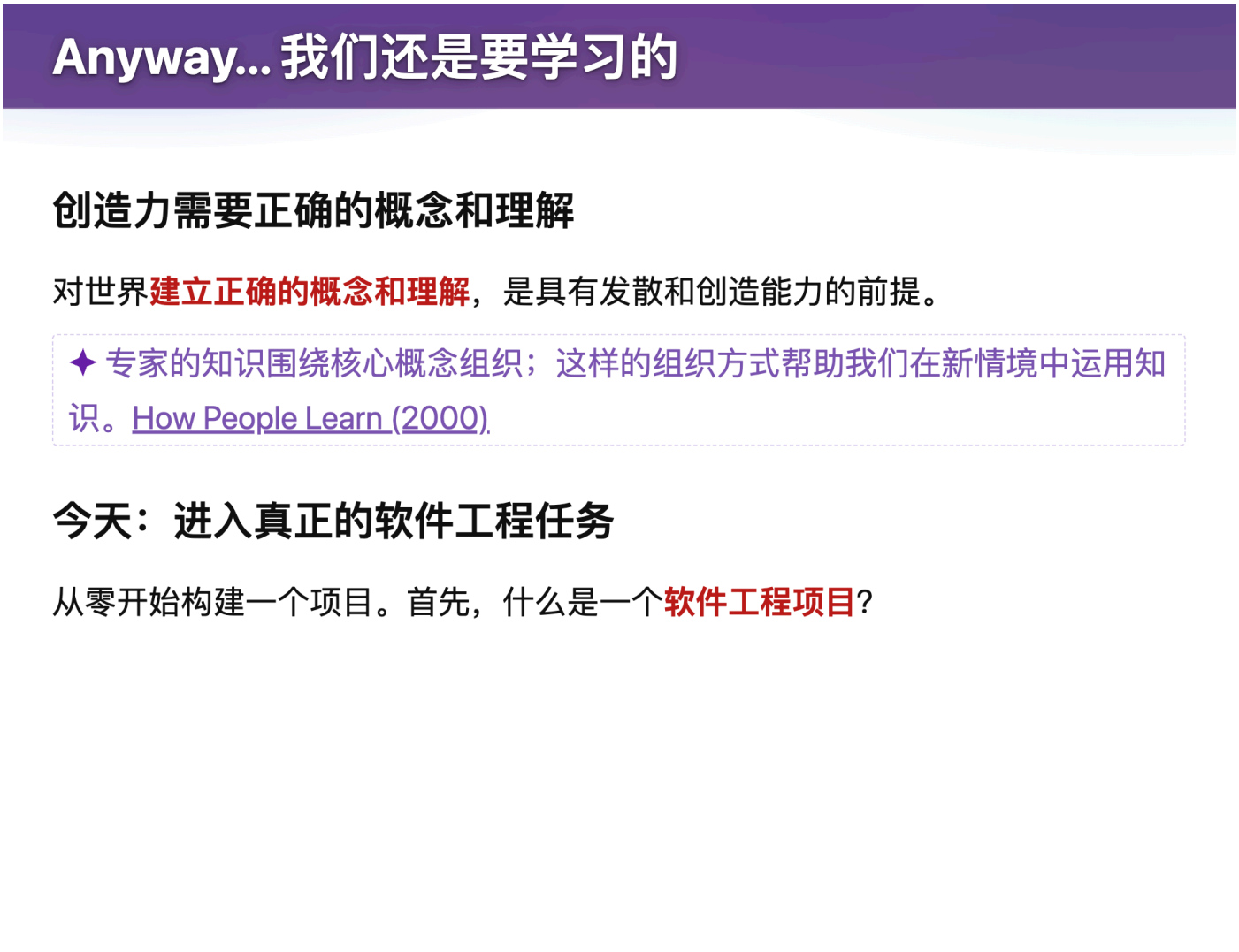
---

# AI 时代还要学基础吗：概念决定可以借鉴什么

> **关键判断 · 整理者提炼**
> 基础知识提供正确概念，让人能借鉴设计、提出问题并判断边界。

讲师借汽车作比喻：早期司机常需懂机械原理才能排除故障，后来人可以在不了解整个系统的情况下驾驶。AI 也会逐步把某些底层操作包起来，因此未来究竟该怎样学习，他明确承认还没有确定答案。

在当前阶段，他仍认为**操作系统**、数据库等知识值得学。AI 扩大执行力，但人需要**正确概念**，才能把别处的好设计借过来、提出更好的问题、判断方案的边界。






**资料图 · 第 4 页** 创造力需要正确的概念和理解；当前学习的目的不是只记操作。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 4 · 原始讲义 · 创造力需要正确的概念和理解；当前学习的目的不是只记操作。

本课并不是让人背更多 Git 命令，而是从“我想保留过去”“我想和别人一起改”等需求出发，让新概念从已有知识中自然长出来。

第 04 节

00:09:02—00:13:41 · 课堂时间

# 项目是什么：目录、构建和真正的交付

> **关键判断 · 整理者提炼**
>
> 一个项目应把需求、源码、构建和部署连起来，本机运行只是交付过程中的一个阶段。

开始一个项目，最朴素的动作是新建目录。目录里的代码、需求文档、接口说明、测试与构建脚本共同构成**可交付的软件**；它们混合了承载意图、规格与实现的不同材料。




5 / 35

**资料图 · 第 5 页** 一个目录中的 intent/spec/impl，经 build 形成应用或可执行文件。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 5 · 原始讲义 · 一个目录中的 intent/spec/impl，经 build 形成应用或可执行文件。

讲师把软件理解成人类世界需求在信息世界的投影。项目的结果可能是 npm run dev 启动的本地网页，也可能是用 GCC 从 hello.c 编译出的可执行程序。能在自己电脑里运行，是一个具体阶段。

课堂用“做出很多 killer 产品，地址却都在 localhost”的笑话说明知识的重要性：本机地址不等于别人可访问的产品。如果目标是让全世界的人使用，就应在意图和**验收条件**中讲清发布范围，不能假设模型自动替你补上。

从 IDE 的一键运行到理解 build 目录由哪些命令生成，是建立项目模型的过程。你需要区分**源码**、工具、**构建产物**和部署结果，而不只看到“窗口打开了”。

---

第 05 节

00:13:41—00:16:58 · 课堂时间

# 从优盘与邮件，到可恢复的项目历史

> **关键判断 · 整理者提炼**
>
> 保存当前文件只是起点，版本管理还要支持历史恢复与共同接收变化。

讲师的第一位博士生大一时把代码放在 U 盘上，方便在机房与自己的电脑间切换；后来 U 盘损坏，早期作业随之丢失。课堂把这段经历与打压缩包、发邮件、网页上传作业的方式并列，说明“存着当前文件”并不等于**管理好项目**。




6 / 35

**资料图 · 第 6 页** U 盘单份保存与打包发邮件的局限。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 6 · 原始讲义 · U 盘单份保存与打包发邮件的局限。

Git 除了保存目录，还能保留、比较和**恢复历史状态**。课程发布更新时，学生可以 clone 获取项目、pull 接收后续变化，不必反复给每个人发整包。

讲师的 **make submit** 会调用课程脚本打包上传；这是项目定义的**工作流入口**，不是 Git 内置命令。它把版本、构建和交付联系起来，也引出以后会讲的 CI/CD。




7 / 35

**资料图 · 第 7 页** Git 管理版本；make submit 是课程项目约定。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 7 · 原始讲义 · Git 管理版本；make submit 是课程项目约定。

---

第 06 节

00:16:58—00:22:18 · 课堂时间

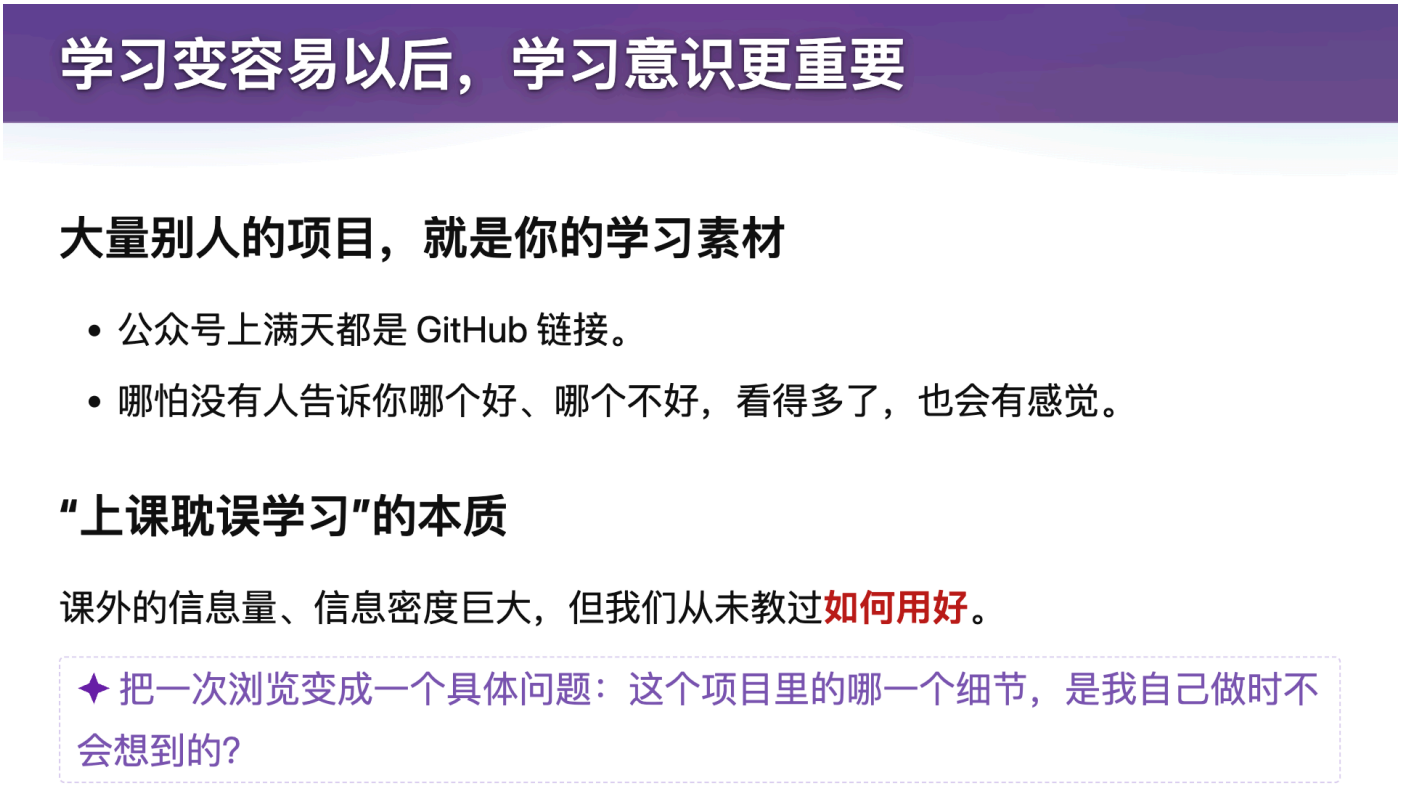
## 从好 README 和路牌中学习设计：在犹豫处给信息

> **关键判断 · 整理者提炼**
>
> 好的文档在读者需要作决定的位置提供下一步，设计判断来自对具体细节的比较。

学习资源容易取得以后，最稀缺的可能是**主动留意**。打开作业仓库只完成指定动作，和比较不同项目为什么这样组织文件、为什么文档这样写，得到的学习结果不同。

讲师比较两种 README：一种非常长，堆满 AI 常见的字符框图；另一种简短清楚，看完后通过少数链接就能到达所需材料。好文档不是追求最短，而是在读者需要作决定时给出**下一步**。

他借香港与南京地铁找路的个人经历解释这个原则：自己在路口产生不确定性时，理想的指示牌正好在面前，直接说明各出口去向。移到 UI 上，就是站在**用户当时的位置**，理解不同目标，再以尽量少的信息把选择呈现出来。




8 / 35

**资料图 · 第 8 页** 把一次浏览变成具体问题：哪个细节是我自己做时不会想到的?

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 8 · 原始讲义 · 把一次浏览变成具体问题：哪个细节是我自己做时不会想到的?

成熟项目可能经历过多年维护、失败和重构；今天看到的细节是经验沉淀。读者可以拿两个实例问 AI：为什么这个写法更容易使用?这比无目的浏览更有机会把别人经验变成自己的判断。

---

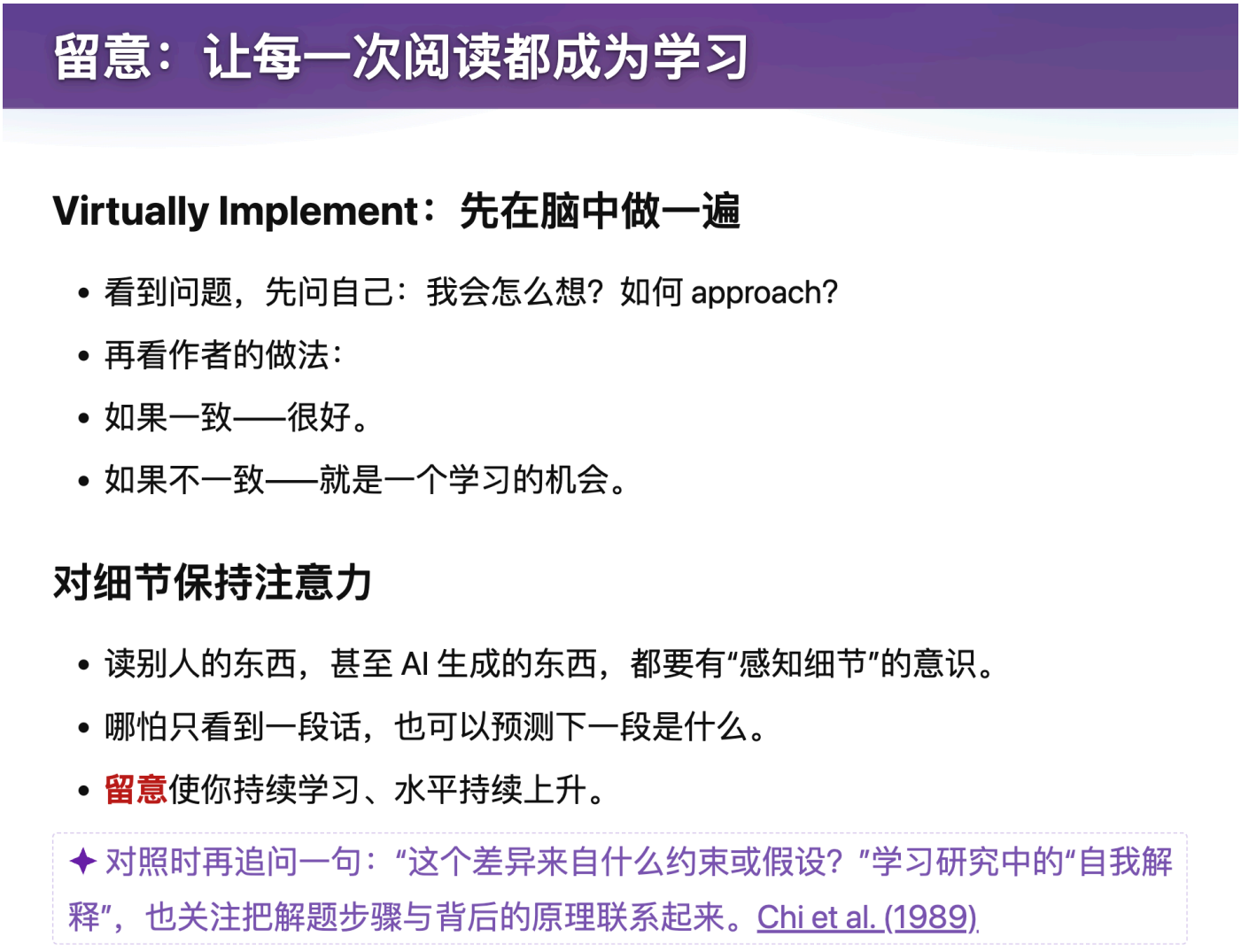
第 07 节

00:22:18—00:24:43 · 课堂时间

# Virtually Implement：先自己想，再对照别人

> **关键判断 · 整理者提炼**
> 先独立构想，再解释与成熟方案的差异，才能把阅读变成可迁移的理解。

读一个方案前，先暂时忽略现成答案，在脑中做一遍：如果这个问题交给我，我会选什么表示、步骤和约束？随后把自己的方案和作者比较，这就是 virtually implement。




**资料图 · 第 9 页** 先在脑中实现，再解释方案差异；讲义同时提示自我解释。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 9 · 原始讲义 · 先在脑中实现，再解释方案差异；讲义同时提示自我解释。

相同的地方可以核对**细节与边界**；不同的地方正是**学习入口**。若成熟方案更精巧，追问它利用了什么自己没想到的条件；若自己的方法似乎更好，也应验证，而不是只凭感觉下结论。

这一方法适用于论文、算法、系统设计，也适用于 AI 的输出。甚至读一段话时预测下一段，再观察对方为什么转向别处，都在训练对细节和逻辑关系的注意。

讲师把这种持续比较视为学习的正反馈。它不等于“想法一致就已经掌握所有实现”；真正理解仍需能解释条件、**处理反例**并用于新场景。

---

第 08 节

00:24:43—00:29:15 · 课堂时间

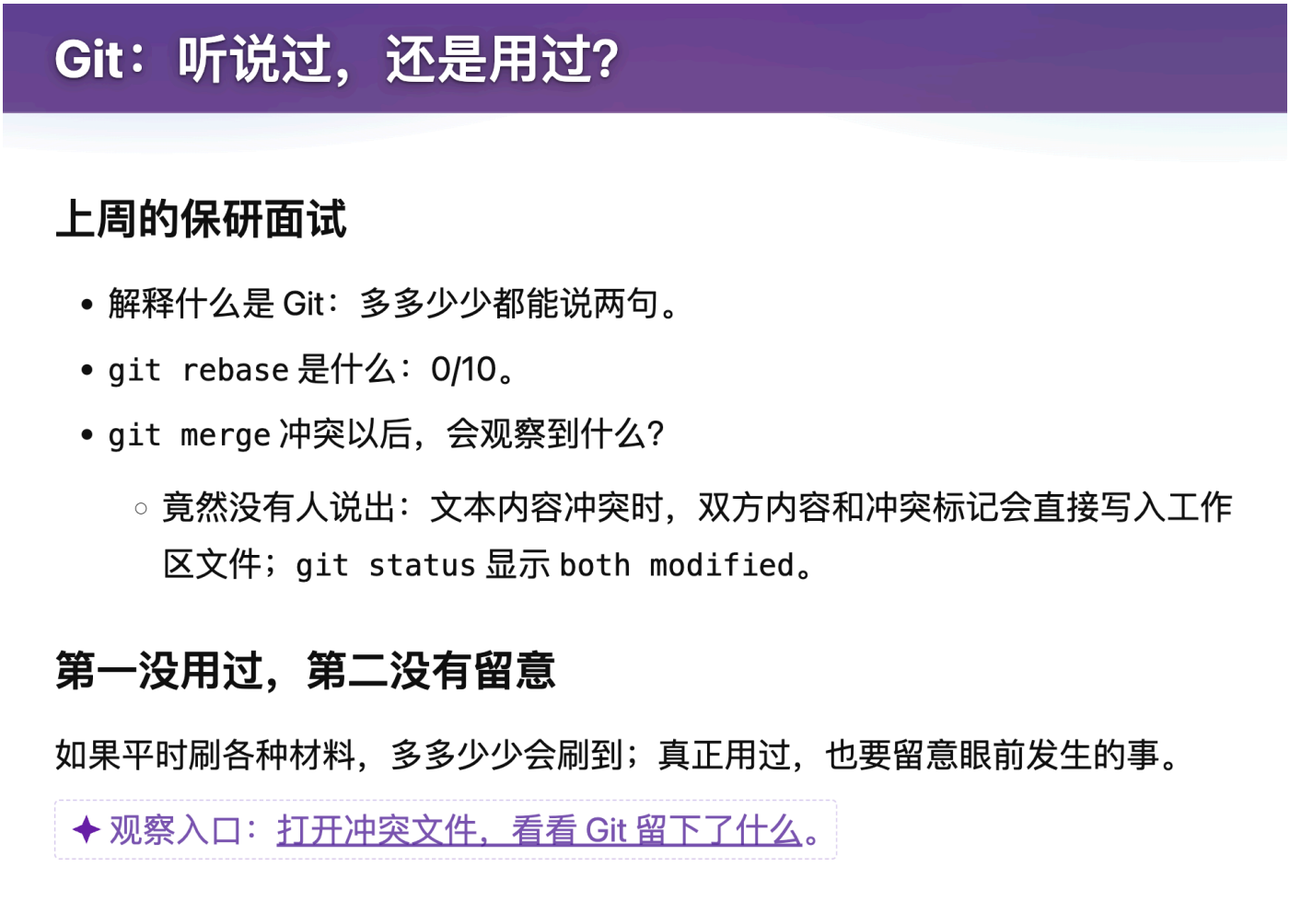
# 合并冲突会出现什么：从使用现场认识责任

> **关键判断 · 整理者提炼**
>
> 冲突标记把待决定的内容带到开发者面前，但工具不能替人作业务选择。

讲师在推免面试中问过 Git、rebase 与 merge 冲突。课堂说十位被问到 rebase 的同学都未答出来，冲突的可见表现也少有人描述；这是一次面试经历，不是对所有学生的统计调查。

以同一行原来为 1、一人改为 2、另一人改为 3 为例，Git 不能代替**业务选择**。冲突后责任回到开发者：工具应把两边内容放在需要决定的地方，降低寻找上下文的成本。

常见**文本冲突**会出现 <<<<<<<、=======、>>>>>>> 等标记，IDE 可能提供接受某侧变更的按钮。开发者要结合需求编辑最终内容，再**标记已解决**并完成合并。




**资料图 · 第 10 页** 面试经历和文本冲突的观察入口。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 10 · 原始讲义 · 面试经历和文本冲突的观察入口。

> **整理者说明**
> 讲义补充：同时看 **git status** 与文件；二进制、删除/修改等冲突不一定都呈现相同文本标记。

这与前面路牌的原则相同：在不确定性发生处提供所需信息。让 Agent 帮忙解决很方便，但学生可以先停一秒，看看工作区和状态实际如何变化，建立下一次预测的能力。

---

第 09 节 00:29:15—00:34:46 · 课堂时间

## 从目录副本推导项目快照：历史模型与存储分开

> **关键判断 · 整理者提炼**
> 版本恢复需要一致的项目状态，而逻辑快照与底层存储方法应分开理解。

最直觉的版本管理是复制目录，命名作业–v1、v2、最终版。能找回旧文件是优点；但版本关系、配套配置和多人同步都要靠人记。若把这些需求交给一个工具，就开始自然推导版本管理。

# Git 之前，版本怎么管?

## 先把旧的留下来

- ✦ 最直接：复制整个目录，叫 作业–v1、作业–v2、作业–最终版。
  - ✦ 能找回旧文件；但“哪份从哪份改来、谁改了什么”，要靠人记。
- ✦ 用工具记：RCS 等工具替你保存文件的历史版本。

## 再让大家一起改

- ✦ CVS、SVN：把历史放在一台服务器，大家取出文件、修改，再交回去。
- ✦ Git：保存项目的一个个版本，也记录它们的来路，帮助大家汇合改动；
- 通常每个人本地就有完整历史。

✦ 版本管理的几种做法 · Pro Git



**资料图 · 第 11 页** 复制目录、文件历史与多人协作，是不同层次的需求。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 11 · 原始讲义 · 复制目录、文件历史与多人协作，是不同层次的需求。

我们真正需要的是**项目级回溯**：代码和配置曾经作为一个整体工作，恢复时也应得到那个**一致状态**。文件删除后，只要内容保存在仍可达的历史中，就可以从旧版本取回；尚未提交的编辑不能假设总能救回。

# 我们要的是项目级“回溯时间”

## 人和 Agent 都会犯错误

- 设计会被改掉，对的代码会被改错……
- 哪怕手滑把文件删掉了，只要已保存在 Git 历史中，就能捞回来。

## Git 管理的是项目的快照

- ✦ 一次提交，记下纳入版本管理的文件当时的内容，以及它们在目录里的位置。
- ✦ 为什么要以项目为单位？代码和配置一起改过，只恢复旧代码、留下新配置，可能拼出一个从未正常运行过的版本。
- ✦ “存过盘”不等于“存进历史”：希望将来回到的状态，需要提交保存。

✦ 快照与提交 · Pro Git



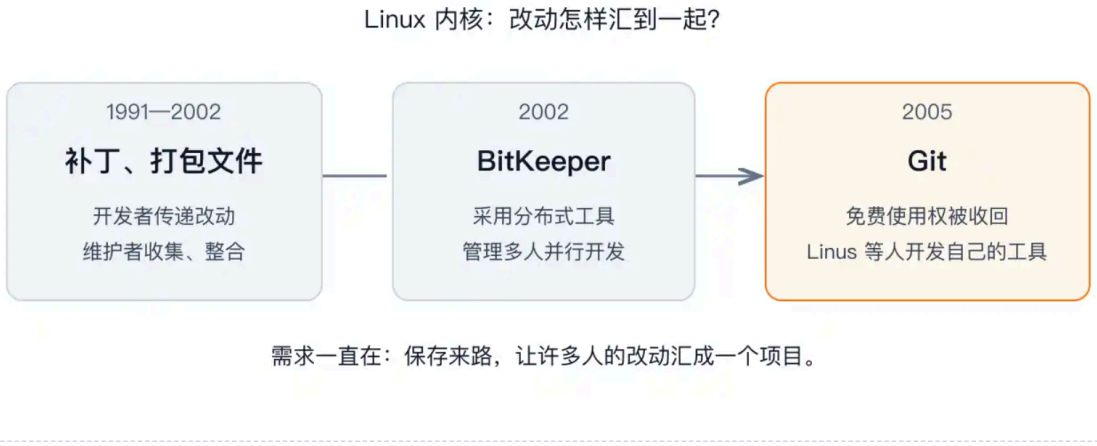
**资料图 · 第 13 页** 项目级快照保存配套状态；存盘和提交不同。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 13 · 原始讲义 · 项目级快照保存配套状态；存盘和提交不同。

课堂回顾 Linux 内核从邮件补丁、BitKeeper 到 2005 年 Git 的背景，并用“**快照**”对比逐个补丁的直觉模型。关键应落在用户可以访问完整项目状态、分叉与汇合的语义上。

# Git 从哪里来?

## 一个大项目，逼出了自己的工具




Git 简史 · Pro Git



**资料图 · 第 12 页** 官方讲义的 Git 简史图。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 12 · 原始讲义 · 官方讲义的 Git 简史图。

> **整理者说明**
>
> 整理者边界：课堂对 SVN/delta 的对比是简化讲法，不能据此断言其他版本系统都必须逐条回退全部补丁。逻辑上的**快照**与磁盘如何压缩、共享数据是不同问题。

讲师鼓励从“我想撤销”“几个文件必须配套”“室友也会修改”逐步外推概念。这比先死记分支、提交等词，更容易理解工具为何要这样设计。

# 从已有经验，长出 Git 的概念

## “现在的你”是一个知识网络

学习的方式，是自然地“外推”它，这样可以帮你记忆。

- ✦ 你知道改错了想撤销 → 项目也需要能回去的旧状态。
- ✦ 你知道几个文件要配套 → 应当记住项目在同一时刻的样子。
- ✦ 你知道多人会同时修改 → 需要知道各自从哪里出发，才能汇合改动。

## 让“快照”成为推出来的需求

✦ 试着预测：删除一个已经提交过的文件，再提交一次，旧版本里还能找到它吗？能——新快照里没有它，旧快照仍然记着它。



**资料图 · 第 14 页** 由已有经验外推快照、历史与协作。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 14 · 原始讲义 · 由已有经验外推快照、历史与协作。

---

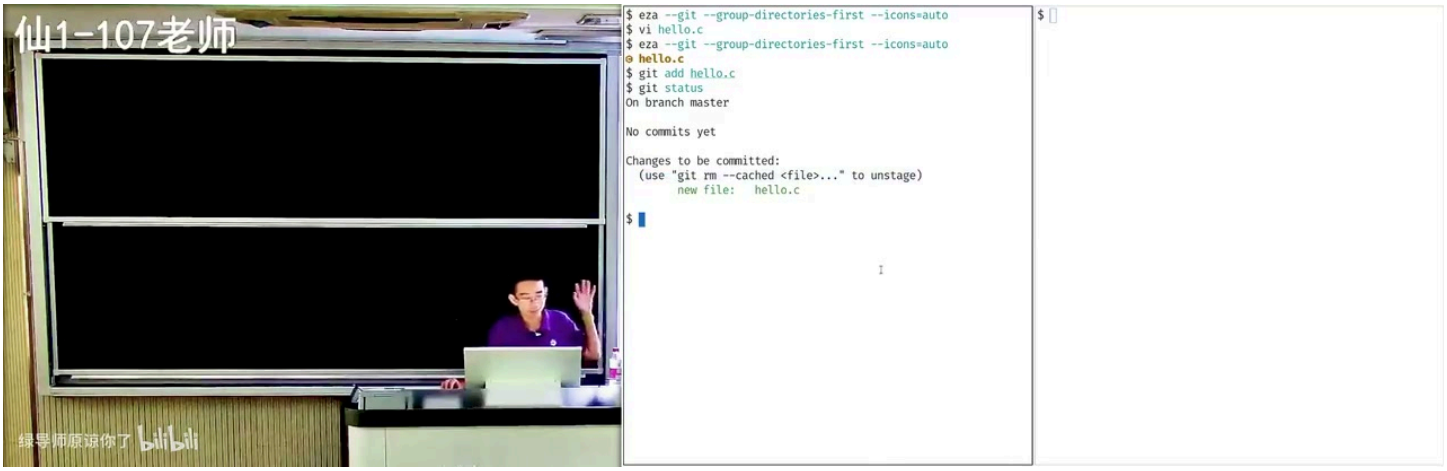
第 10 节

00:34:48—00:40:51 · 课堂时间

# 现场 init → add → commit：提交信息要解释变化

> **关键判断 · 整理者提炼**
>
> 提交需要保存恰当的暂存内容，并解释这次改变做了什么、为什么做。

讲师在空目录里 git init，得到隐藏的 .git 目录，再创建 hello.c、git add 并查看 status。画面显示文件进入 Changes to be committed，而仓库还没有提交。




**视频原帧** 现场：hello.c 已暂存，git status 显示还没有提交。

视频原帧 00:36:30 · 现场：hello.c 已暂存，git status 显示还没有提交。

普通 git commit 将**暂存区**内容保存为快照，配上**提交说明**。Git 鼓励围绕一个可解释的修改组织提交，如修 bug、增加功能、调整脚本或文档；不只是按作业-v1/v2随意保存一团变化。

> **整理者说明**
>
> 讲义补充：add 后再改文件，普通 commit 采用**已暂存版本**；git diff 和 git diff --cached 分别观察未暂存与已暂存的变化。

# 从空白开始

## Tutorial 告诉你的基本模型

`git init` 会得到一个空的 repository (repo)。可以使用一些不知道什么意思的命令，和 GitHub、GitLab、……建立关联。

- `git clone`：从远处下载代码
- `git add`：可以“添加”到 Staging Area (Index)
- `git commit`：可以“提交”一个快照

## 就可以开工了?

✦ `git add hello.c` 之后，又改了 `hello.c`，下一次 commit 会提交哪个版本?

✦ 提交的是 **add 时暂存的内容**。会运行一个命令，和知道它改变了什么，是两回事。Git 官方说明

15 / 35

**资料图 · 第 15 页** 工作区、暂存区与提交：add 保存的是当时的内容。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 15 · 原始讲义 · 工作区、暂存区与提交：add 保存的是当时的内容。

他故意使用 I did some cool stuffs 作为说明，再问 Agent 是否符合 best practice。AI 的核心批评是读不出**改了什么、为什么改**。若上万条提交都这样，说明字段就失去帮助别人理解项目演进的价值。




**视频原帧** 现场 Agent 对含糊提交说明提出批评；表格是模型输出，不等于每项判断均正确。

视频原帧 00:39:40 · 现场 Agent 对含糊提交说明提出批评；表格是模型输出，不等于每项判断均正确。

> **整理者说明**
>
> 整理者提醒：2380 秒表格还批评空 main 的 include/return；整理者用本机编译器以 C99 对 int main() {} 做语法检查可通过，因此不能把“没有 include/return”直接等同编译错误。此小检查不验证整个课堂工程。

消息应能让维护者顺着历史看到需求增加、修复和重构的轨迹。课堂显示 AI 会给表格和改写建议，但建议本身仍要核对，不能因为语气笃定就全部当成规则。

---

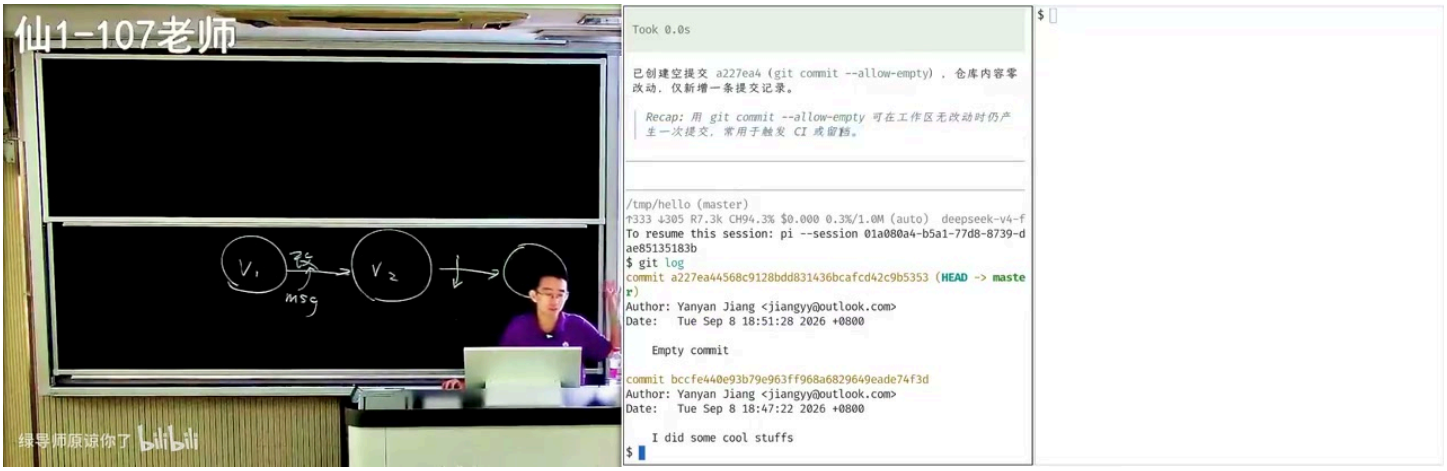
第 11 节　　00:40:51—00:44:54 · 课堂时间

# 空提交、标签和发布：能力不等于默认做法

> **关键判断 · 整理者提炼**
>
> 提交还记录元数据和历史关系，允许空提交不意味着它应成为默认做法。

讲师继续追问：两个快照内容完全相同，能不能仍有新提交？普通提交在没有变更时拒绝，Agent 随后用 **git commit --allow-empty** 建立 Empty commit。现场日志出现新记录，但仓库内容没有变化。

> 案例回扣　2548 秒画面确认命令 --allow-empty，日志同时保留 Empty commit 与此前 I did some cool stuffs。




**视频原帧**　现场空提交成功：新记录存在，但没有对应的文件内容修改。

视频原帧 00:42:28 · 现场空提交成功：新记录存在，但没有对应的文件内容修改。

这帮助区分提交对象与文件内容：提交还包含**元数据与历史关系**。空提交在触发某些 CI 流程或记录特殊节点时可能有用，但无意义堆积会降低历史可读性。

他以最后一次 bug fix、版本冻结和 release 作例子：可以用**标签**指向已完成版本，也可以有意安排一个空提交记录某种流程事件。该如何选，取决于**项目约定**，而不是“命令允许就处处使用”。

表面上学会 add/push/pull 就能工作，水面下仍有大量实践知识。让 AI 解释某个具体决定为什么这样做，可以把隐含经验逐渐变成自己的判断。

---

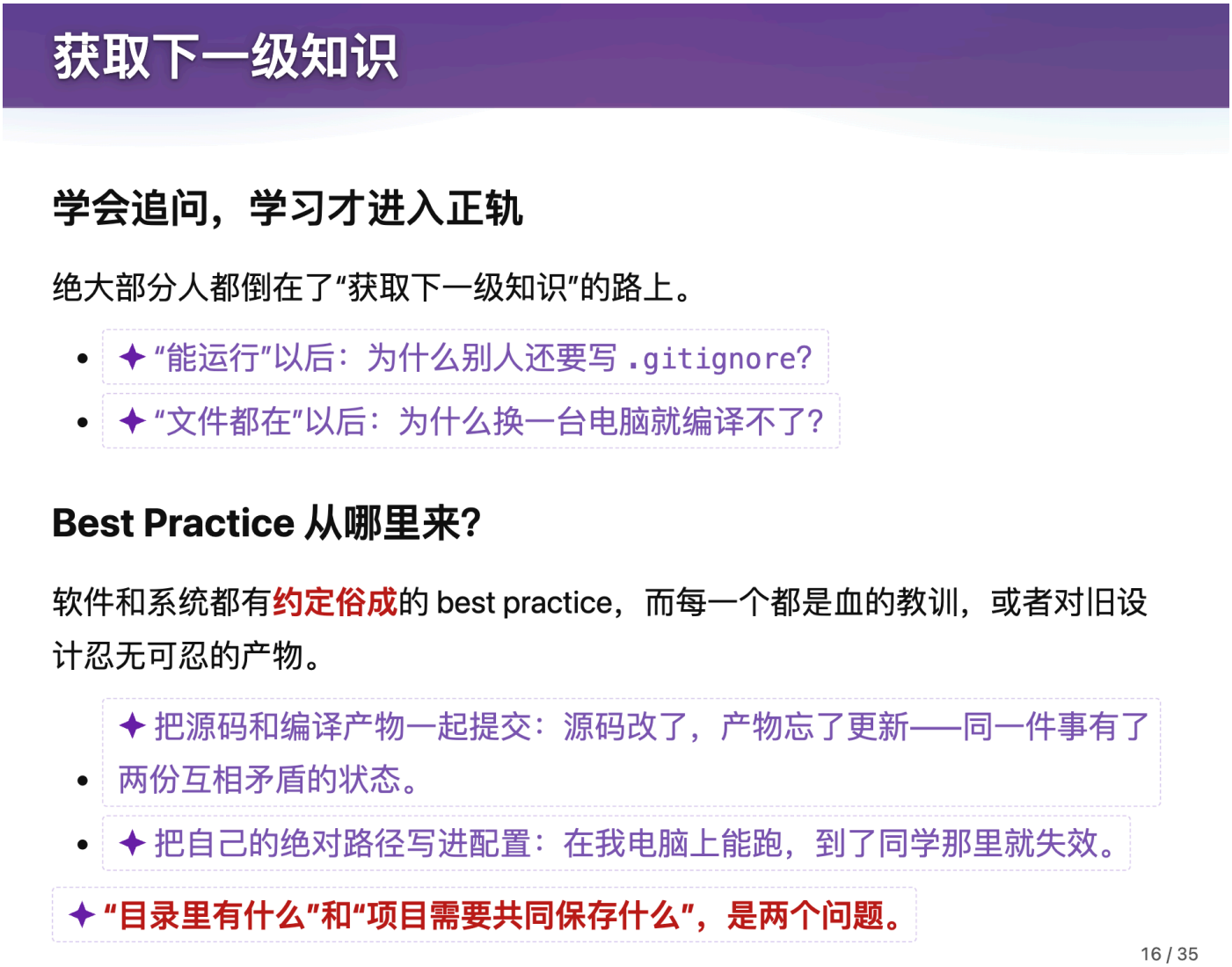
第 12 节

00:44:54—00:51:13 · 课堂时间

# git add . 的代价：构建产物与整理历史的前提

> **关键判断 · 整理者提炼**
>
> 共享源码和构建方法比随手提交所有文件更可靠，整理历史前要确认协作者是否依赖旧版本。

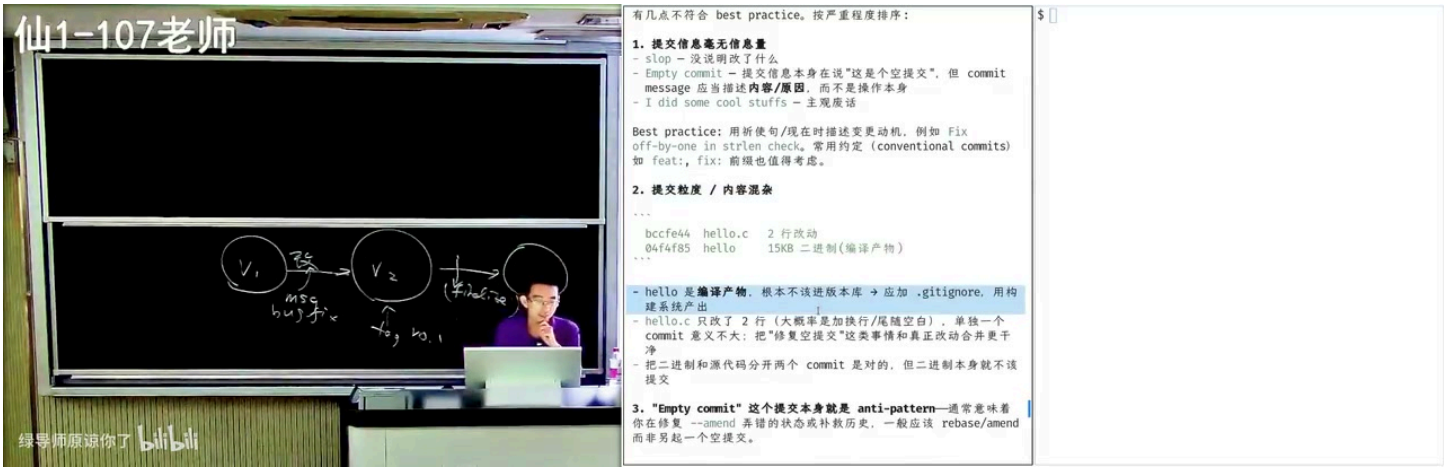
讲师编译 hello.c 得到 hello，再用 git add . 一次加入目录变化。Git 允许这样提交，Agent 随后的审查却指出：说明无信息、不同性质的变化混杂、**编译产物**不宜进入该仓库。




**资料图 · 第 16 页** “目录里有什么”和“项目需要共同保存什么”是两个问题。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 16 · 原始讲义 · “目录里有什么”和“项目需要共同保存什么”是两个问题。

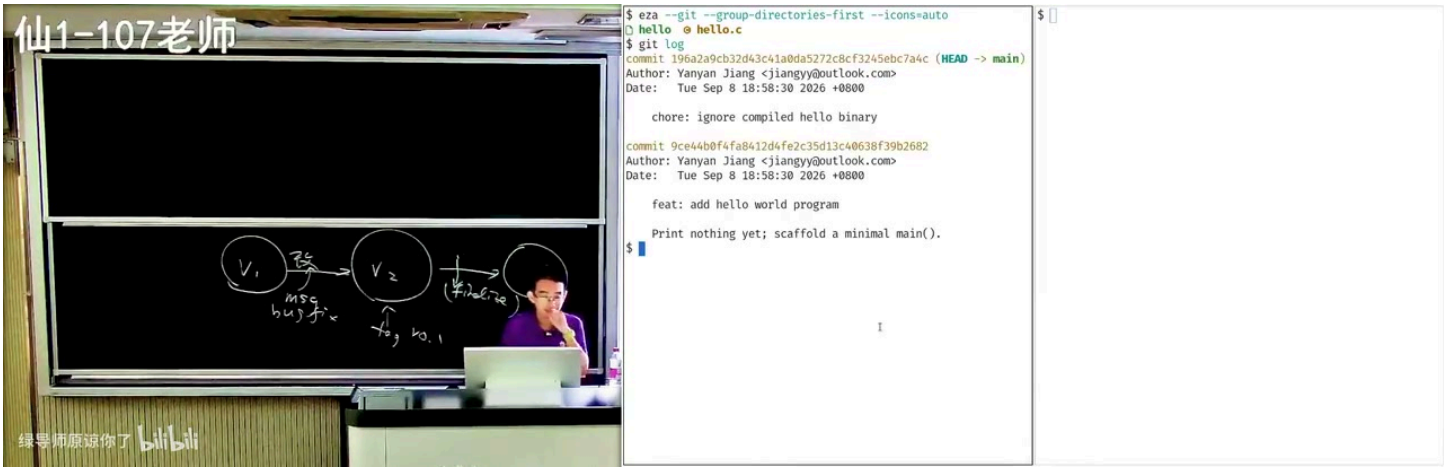
**构建产物**通常能由源码和构建方法**重建**；不同机器或构建参数可能生成不同二进制，即使源码相同也可能造成无意义差异与难以合并的冲突。因此需要说明如何生成它，并用忽略规则保护日常操作。




**视频原帧** Agent 指出 hello 是编译产物，建议源码与构建方法共同维护。

视频原帧 00:46:55 · Agent 指出 hello 是编译产物，建议源码与构建方法共同维护。

现场 Agent 发现这是没有 **remote** 的练习仓库，重写成两个较清楚的提交：feat: add hello world program 与 chore: ignore compiled hello binary。正文说明与标题分开，历史更容易读懂。




**视频原帧** 整理后的本地练习历史：程序骨架与忽略编译产物各有清楚意图。

视频原帧 00:49:26 · 整理后的本地练习历史：程序骨架与忽略编译产物各有清楚意图。

讲师强调其检查 **remote** 的细节：若其他人已依赖原历史，重写会给协作者添麻烦。没有 remote 只是课堂练习情境中的线索；实际还应确认是否有人通过其他方式依赖了**旧提交**。

> **整理者说明**
>
> 课堂提到提交消息可关联 issue；自动关闭是否发生还取决于平台规则、关键词和合入分支，不应理解为任意文本出现 issue 号都会关闭。

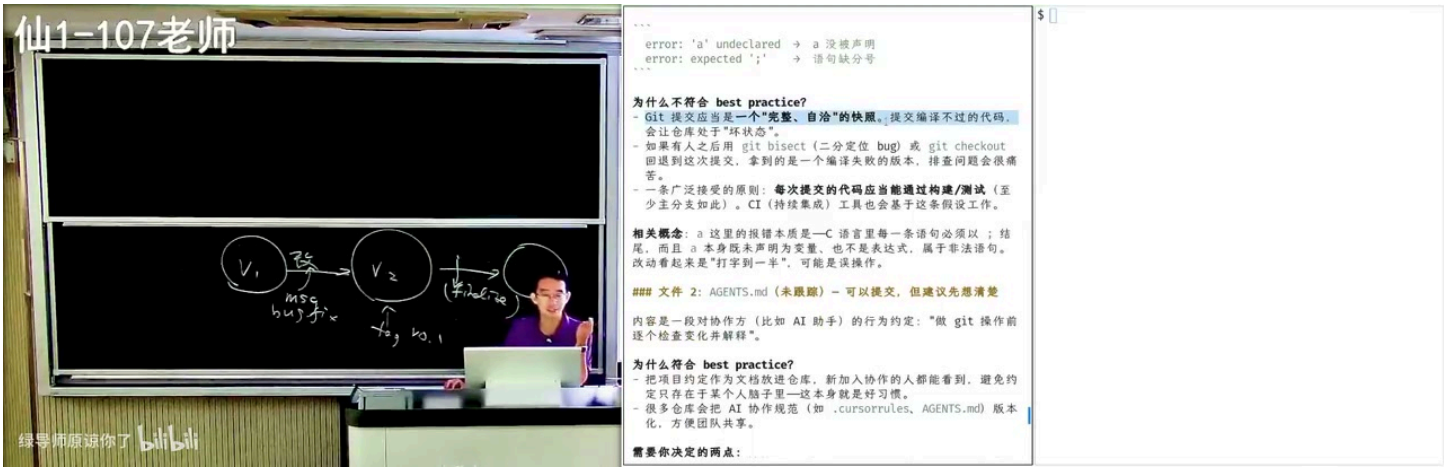
---

第 13 节 00:51:13—00:55:13 · 课堂时间

# 可构建的提交为什么重要：bisect 与 blame

> **关键判断 · 整理者提炼**
>
> 可构建、可测试的版本使 bisect 能定位故障，blame 则提供继续回查的线索。

讲师故意把 hello.c 改成不能编译，再让 Agent 提交。Agent 指出未声明标识符与缺少分号，建议先修正，并解释提交应形成**完整**、**自洽**的状态。




**视频原帧** 现场：Agent 识别无法编译的修改，并解释自洽快照与 bisect 的关系。

视频原帧 00:52:20 · 现场：Agent 识别无法编译的修改，并解释自洽快照与 bisect 的关系。

若每个候选版本都能构建、测试，历史才能用于定位故障。**git bisect** 的课堂直觉是：已知一个好版本与坏版本，测试中间版本，反复缩小范围，找出引入问题的变化。

讲师用“前缀正常、后缀出错”的数组解释二分。在真实历史中，应有明确**可复现的判定**；不能编译或无法判断的版本会增加定位难度，测试不稳定也会破坏判断。

**git blame** 提供每行代码的历史出处，帮助继续查提交说明与上下文。它是理解来源的线索，不是单凭名字判定责任或根因的证明。

---

第 14 节

00:55:14—00:59:37 · 课堂时间

# 哪些文件进入仓库：判断用途，不机械看文件名

> **关键判断 · 整理者提炼**
>
> 是否跟踪文件取决于共享用途和可重建性，不能机械地按隐藏或生成文件分类。

讲师列举新成员常见问题：dist、运行日志、个人编辑器设置、机器绝对路径、API key、**.DS_Store** 等被一起提交。它们可能增加噪声、破坏**可移植性**或暴露不应共享的信息。

点开头只是隐藏文件约定，不等于 Git 自动忽略。课堂在 macOS 下示范 **.DS_Store** 隐藏却会被 git add . 跟踪；另一方面 .github/workflows 等点目录又确实属于项目配置。

规则有例外：**package-lock** 等锁文件虽然由工具生成，却能固定依赖；生成的头文件有时有意保留，使使用者不必安装另一套生成工具链即可编译。判断点是**用途与可重建性**，而非“生成文件一律不提交”。

# 进组以后的高血压时刻

## 我和许多研究生新生都说过

- 生成的文件不提交 (`dist/`、log、……)
- 个人的配置不提交 (`.vscode` 中的个人设置等)
- ✦ 判断的是用途：团队共享、可移植的配置可以提交，不能只看目录名。
- 密钥不能提交 (审稿的时候见过好多)
- 临时文件不提交 (`.DS_Store`、……)
- 规则是活的：package lock, generated header, ...

## 忽略不等于遗忘

- ✦ `.gitignore` 不会让已经被跟踪的文件自动退出版本管理。
- ✦ 密钥一旦泄露，删掉当前文件也不能收回别人拿到的副本；应先撤销或轮换凭据，再处理历史。GitHub 指南



**资料图 · 第 18 页** 提交规则有例外；.gitignore 不会自动取消已有跟踪，也不抹掉历史。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 18 · 原始讲义 · 提交规则有例外；.gitignore 不会自动取消已有跟踪，也不抹掉历史。

AI 可以耐心解释每次具体修改，降低初学者被批评的压力，但它并不免除判断。项目共享、可移植的配置可以保留，个人路径与偏好则应隔离。




## 哪怕只是放一个 hello.c

缺少足够的知识，也会犯错误。这时候，对细节的“注意力”就非常重要了。

> AGENTS.md: 在做任何 git 操作之前，都逐个文件检查变化，并为我 (新手) 解释是否符合 best practice，为什么，以及相关的概念。

## 让解释长出下一次判断的能力

- ✦ hello.c：程序的源码，要保存。
- ✦ hello：如果是编译出来的可执行文件，通常不提交；保存源码和构建方法，让别人也能生成它。
- ✦ 接着追问：这个文件是项目的输入，还是构建、运行留下的产物？别人拿到仓库，是否具备重建它的条件?

17 / 35

**资料图 · 第 17 页** 让 Agent 逐文件解释变化，作为学习约定而非自动放行。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 17 · 原始讲义 · 让 Agent 逐文件解释变化，作为学习约定而非自动放行。

> **整理者说明**
> 讲义补充：密钥误公开后，删除当前文件不能收回已有副本；需先**撤销或轮换凭据**，再处理历史。

---

第 15 节 00:59:37—01:03:21 · 课堂时间

# Hook 与 TYPO-CHECK：小设计也能解释理由

> **关键判断 · 整理者提炼**
> 规则标识和自然语言解释承担不同角色，醒目的格式还应服从项目一致性。

Hook 是在某类事件发生时执行动作的机制。课堂既提到 Git 提交前检查，也尝试让 Agent 在需要人输入时播放提示音；反馈并不确定，讲师没有继续修复，所以不能把这一演示写成验收成功的功能。

更严肃项目里有很多检查规则，要选 [TYPO-CHECK] 还是 [typo-check]？两者都可表达规则，讲师直觉大写稍好，却一时说不清原因，于是让 AI 解释。

# 大写还是小写?

## 真实项目中的一个选择

```
[TYPO-CHECK] Docs should be typo-free
v.s.
[typo-check] Docs should be typo-free
```

Check-rule 的名字应该是大写还是小写?

## 直觉也值得追问

我的直觉是“大写似乎更好，但不是极强”，但不太说得出原因。

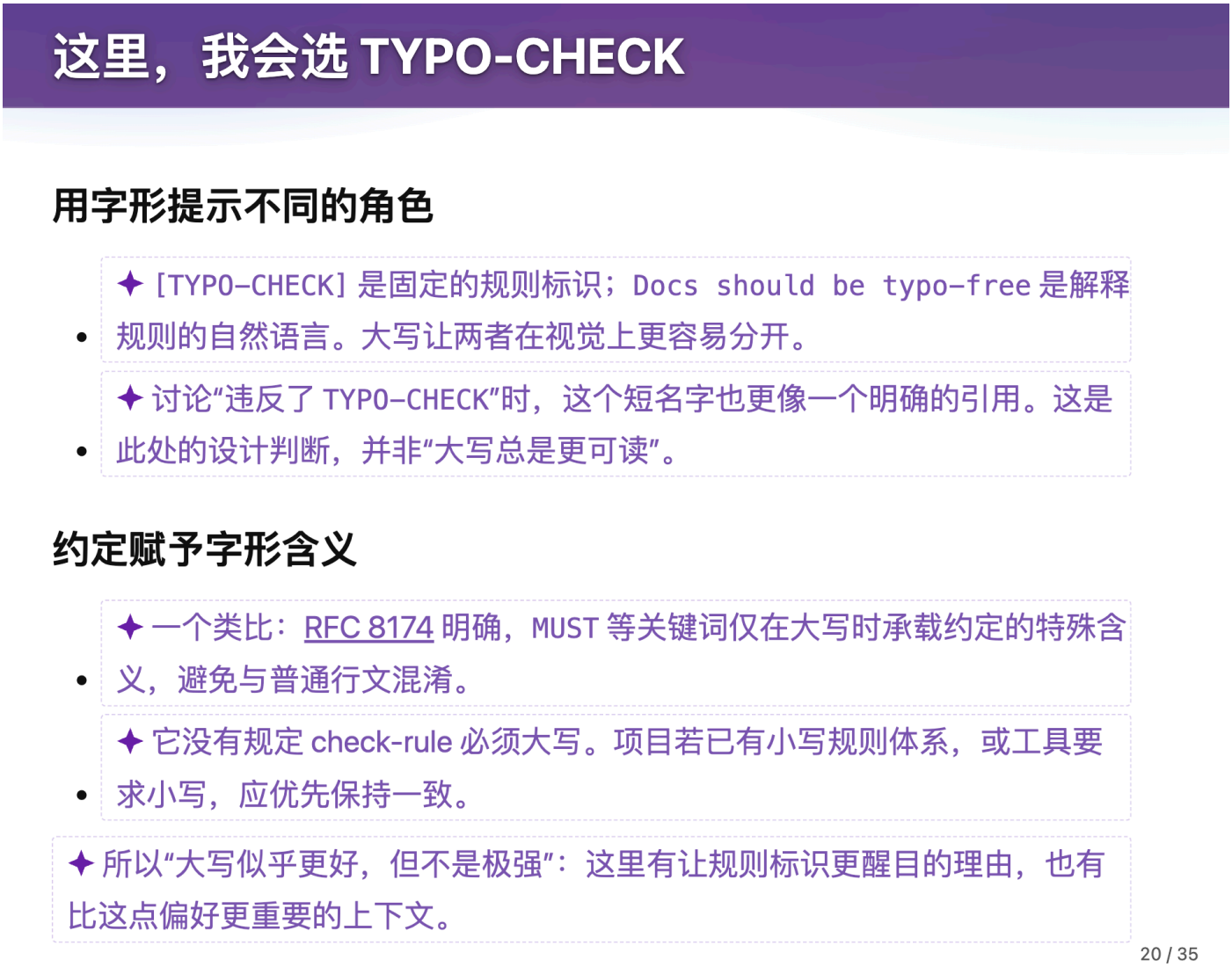
✦ 先想一想：方括号里的字符，是一句话的一部分，还是一个可以被引用的规则名字?



**资料图 · 第 19 页** 两种规则名的选择：先识别名字在文档里的角色。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 19 · 原始讲义 · 两种规则名的选择：先识别名字在文档里的角色。

解释的重点是角色区分：方括号中的短名是可引用的**固定标识**，后面的 Docs should be typo-free 是**自然语言说明**。大写使它们视觉上更容易分开，也方便讨论“违反了哪一条规则”。




**资料图 · 第 20 页** RFC 大写关键词只作类比，不规定检查规则必须大写。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 20 · 原始讲义 · RFC 大写关键词只作类比，不规定检查规则必须大写。

这是语境中的设计取舍。项目已有约定、工具要求或**一致性**，都可能比局部更醒目重要；不能推出“大写永远更好”。

---

第 16 节

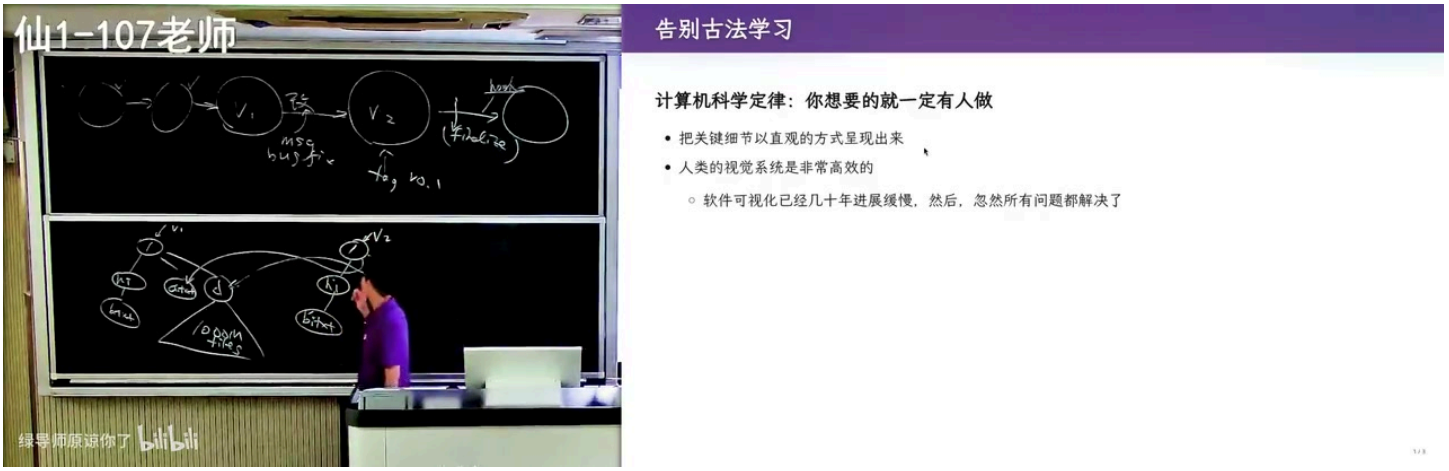
01:03:21—01:08:15 · 课堂时间

## 持久化目录树：快照如何共享没变的部分

> **关键判断 · 整理者提炼**
>
> Git 的完整快照可以复用未改变的子树，只为变化内容及其引用路径建立新对象。

讲师从纸面推导 Git 的底层模型：如果目录树很大，只改 b.txt，保存下一版本不必把所有文件复制一遍。变化的文件需要新内容；从它到根的目录路径需要反映新的引用，其他子树可以继续**共享**。

因此，逻辑上每个版本是**完整快照**，存储上却可以复用未改变的内容。黑板把 V1 与 V2、文件节点、目录节点及**共享**的大子树画在一起，解释“大项目、小修改”为什么可行。




**视频原帧** 现场黑板：修改 b.txt 后复制变化路径，未变子树继续共享。

视频原帧 01:05:50 · 现场黑板：修改 b.txt 后复制变化路径，未变子树继续共享。

.git/objects 中能找到按内容标识组织的对象。课堂重点介绍 **blob**（文件内容字节）、**tree**（目录条目）和 commit（根树与父提交等信息）；这些对象及引用组成可以不断扩展的历史结构。

> **整理者说明**
>
> 整理者说明：这里讲的是用于理解的对象关系，未完整讨论 Git 的 pack 压缩、垃圾回收或全部对象种类。

接下来要检验纸面模型是否真的对应实现：分叉时怎样复用、合并以后出现什么对象、原始字节怎样变成屏幕中的结构。这引出真实仓库的可视化。

---

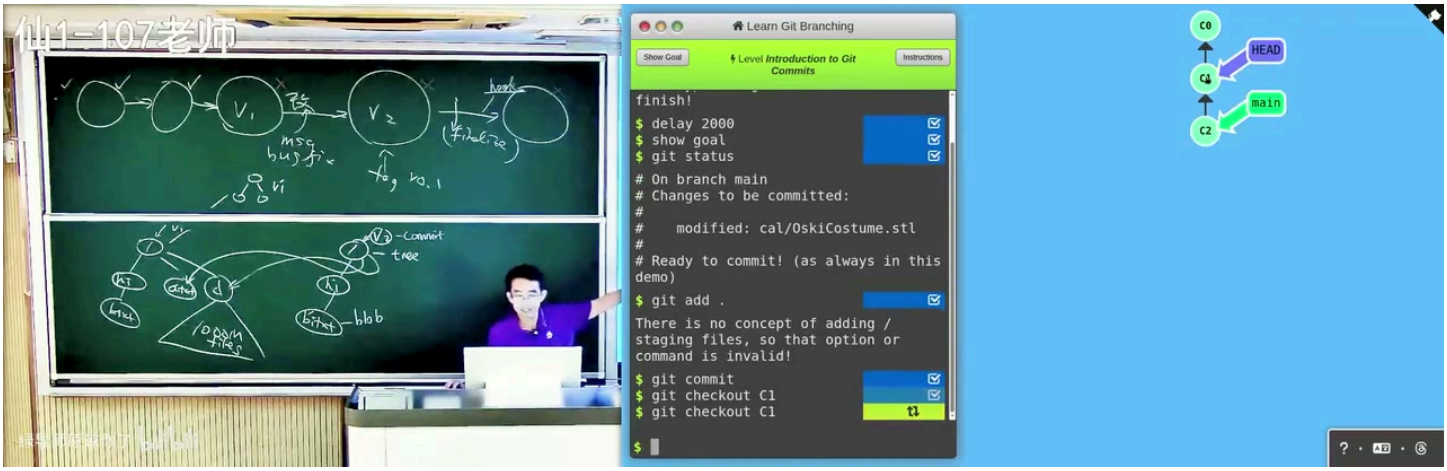
第 17 节

01:08:16—01:11:17 · 课堂时间

# 从模拟到真实仓库：gitv 的需求说明

> **关键判断 · 整理者提炼**
>
> 教学工具的价值在于从真实数据展开关键细节，并让这些关系易于观察。

讲师先打开 **Learn Git Branching**，演示 commit、checkout 后图中版本和引用的变化。它适合学习分支模型，但课堂画面也显示它不支持添加/暂存文件的概念，因此不能期待它完整展示磁盘里的真实 Git。




**视频原帧** Learn Git Branching 的模拟画面：提交/切换有动画，add/staging 显示不支持。

视频原帧 01:09:18 · Learn Git Branching 的模拟画面：提交/切换有动画，add/staging 显示不支持。

## 告别古法学习

### 计算机科学定律：你想要的就一定有人做

- 把关键细节以直观的方式呈现出来
- 人类的视觉系统是非常高效的
  - 软件可视化已经几十年进展缓慢，然后，忽然所有问题都解决了

21 / 35

**资料图 · 第 21 页** 寻找或生成能把关键细节展开的工具。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 21 · 原始讲义 · 寻找或生成能把关键细节展开的工具。

随后展示用 AI 生成的 **gitv**：一个 Node.js 命令行工具，接收真实仓库路径、启动 HTTP 服务、持续监听变化，并在浏览器中可视化对象与引用。核心要求是从真实文件与**原始 bytes** 出发，一步步展示解析结果。

提示词还要求合理空间布局、拖动、大小文件均可读、内容与 diff、实时 CSS 动画、浅色课堂背景；能自行说明的关系不再用多余 tooltip 堆解释。它既约束信息完整性，也约束**使用体验**。

## 把真实的 Git Repo 展开给你看

> 实现 gitv 命令行工具 (nodejs)，把一个**真实**的 git repo (你可以使用 git 命令) 路径里的主要概念彻底、完整地可视化。展示任何对象之前，都从文件系统里的真实文件、原始 bytes 出发，展示一步步如何解析，解析成了什么。通过空间合理的方式排布各类对象，并支持拖动；明快、极简的风格，不要多余的 tooltip text 等，在能够 self-explain 的时候绝不出现文字。大小文件的可视化都要有不错的视觉效果，对于有具体内容的文件，有适当的大小和视觉风格显示内容、diff 等。启动一个 http server，持续监听目录中 git 的变化，并且在发生变化后，用 css 动画效果展示 repo 发生的变化。浅色背景适合课堂展示。

### 直接生成，现场对比

- GPT-6 直出 v.s. https://learngitbranching.js.org/; 国模直出 v.s. https://code.cs61a.org/

### 从看懂到学懂

- ✦ 算法可视化研究的启发：学习者怎样使用可视化，比可视化展示了什么更影响学习效果。Hundhausen 等，2002

22 / 35

**资料图 · 第 22 页** gitv 完整提示词：真实 bytes、对象解析、持续监听与交互要求。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 22 · 原始讲义 · gitv 完整提示词：真实 bytes、对象解析、持续监听与交互要求。

讲师称大体直接生成、只做少量后续调整，现场也承认布局不完美。这是一个可继续迭代的教学工具示例，不能据此断言所有可视化问题已经解决。

---

第 18 节

01:11:35—01:14:23 · 课堂时间

# 逐层拆开 blob 与 tree：字节、解析和链接

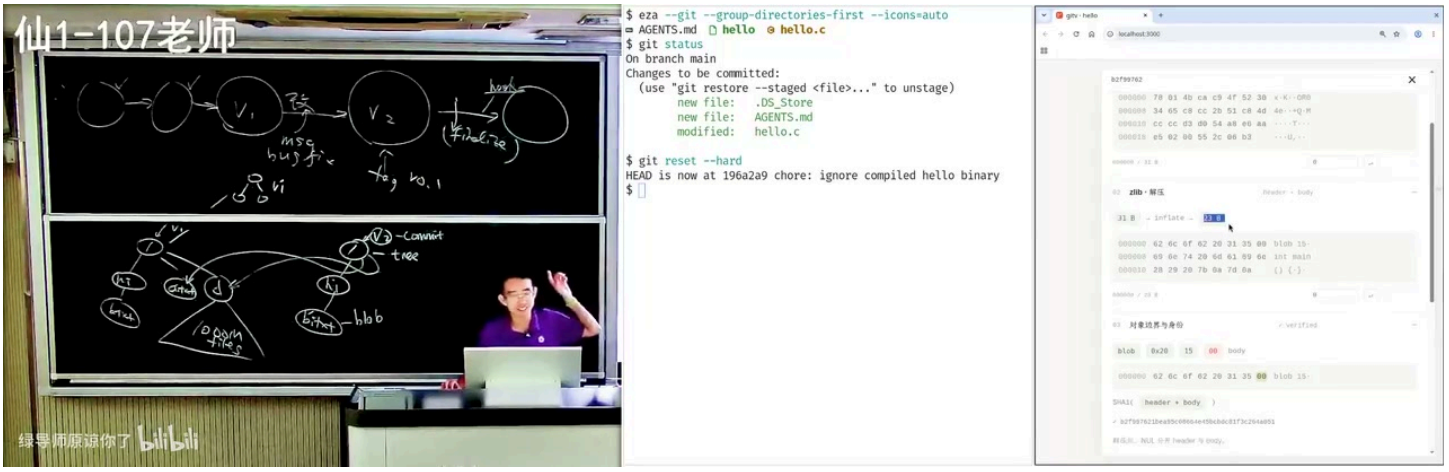
**关键判断 · 整理者提炼**

从磁盘字节逐层解析对象，可以区分文件内容、名称路径与历史关系分别存在哪里。

在真实练习仓库中清理未提交变化后，图上的相应区域同步消失；接着讲师打开旧提交，检查其中 hello.c 的对象内容。画面从磁盘对象的十六进制字节，经过 **zlib 解压**，到**对象头**和正文解释。

> **整理者说明**
> 课堂屏幕出现 reset --hard，是清理该练习仓库的现场记录；它会丢弃部分未提交修改，不作为日常可直接照抄的操作建议。

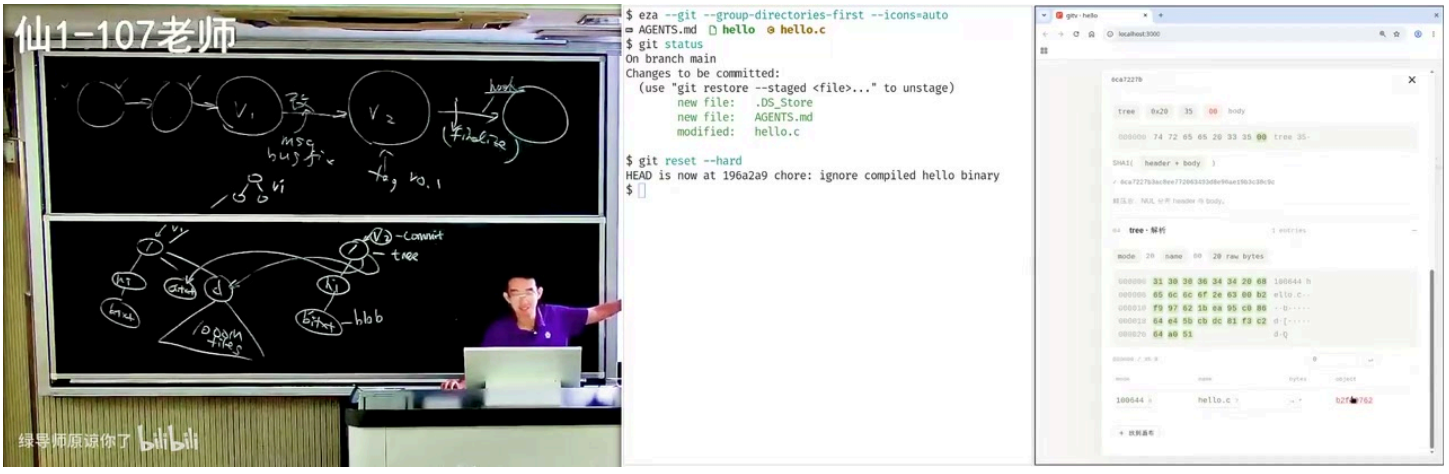
小文件的压缩未必节省空间：课堂此例显示 31 B 的压缩对象解出 23 B，其内含 blob 15 头部和文件正文。这里数字属于演示对象，不是所有 Git 文件的固定开销。




**视频原帧** 真实 blob：原始字节 → zlib 解压 → 对象头和正文，课堂此例为31 B→23 B。

视频原帧 01:12:29 · 真实 blob：原始字节 → zlib 解压 → 对象头和正文，课堂此例为31 B→23 B。

**tree** 不只保存文件内容，还保存名称、**模式和对象引用**。通过 hello.c 与可执行 hello 的对照，讲师解释普通文件和可执行文件在条目模式上的差异，并把图上的连线对应回解析后的对象。




**视频原帧** 真实 tree：解析后的模式、文件名 hello.c 与对象链接。

视频原帧 01:13:29 · 真实 tree：解析后的模式、文件名 hello.c 与对象链接。

> 案例回扣 4349 秒截图可见压缩到解压的字节数，以及 blob 15 正文；4409 秒可见 **tree** 条目解析为 mode 100644、name hello.c 和目标对象。

从抽象概念到真实文件，学习者可以反复检查：哪个对象保存内容，哪个对象保存名字与路径，哪个对象保存历史关系。工具的价值在于降低这种回查成本，而不是替学习者得出所有结论。

---

第 19 节

01:14:23—01:18:09 · 课堂时间

# 分支、冲突 Index 与双父提交：把一次合并走完

> **关键判断 · 整理者提炼**
>
> 分支引用、冲突 Index 和双父提交把一次合并的待决信息与历史来路记录下来。

Agent 从同一基线创建 A、B 两个分支：A 把 main 改为 void 参数，B 引入 argc/argv。两边改动落在同一行，特意为合并冲突准备条件。终端和右侧真实仓库图同步变化。

**分支是指向提交的引用**；普通分支工作方式下，**HEAD** 通常指向当前分支引用。4520 秒画面能看到 refs/heads/A 与目标对象标识，帮助理解“名字”和“被指向的历史状态”不是一回事。

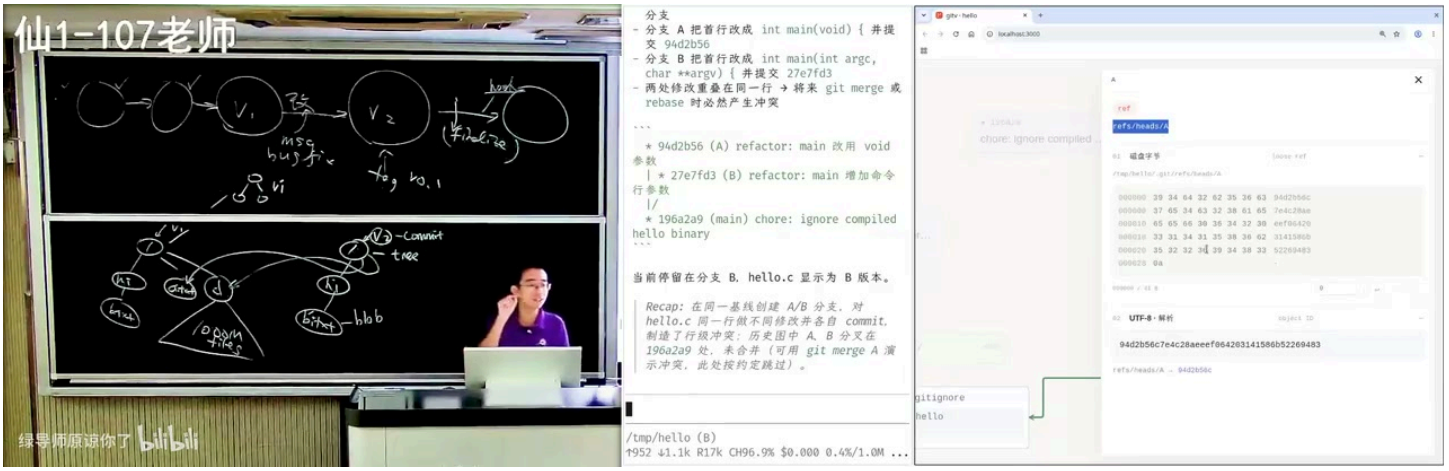
> **整理者说明**
>
> 整理者校正：4530 秒附近口述把引用对象称作 tree；正确关系是分支通常指向 commit，commit 再关联根 tree。正文按官方完整讲义及现场对象关系表述。

官方完整讲义补充

## 讲义扩展：分支、merge、rebase 的完整语义

分支名是可移动的提交引用；**HEAD**在通常分支工作状态中指向分支。merge利用**共同祖先**和双方变化：可快进时移动引用，真正分叉后常形成双父提交。rebase把一串变化重放到新起点，通常生成新提交标识，也可能遇到冲突；别人依赖的历史不能只顾自己整理。此为官方完整讲义的系统解释，视频重点演示了冲突与双父提交，没有完整实操全部rebase过程。

PDF物理页 10、13、15 · 原始资料 ↗




**视频原帧** 现场 refs/heads/A：分支引用指向提交，A/B 从同一基线作出不同修改。

视频原帧 01:15:20 · 现场 refs/heads/A：分支引用指向提交，A/B 从同一基线作出不同修改。

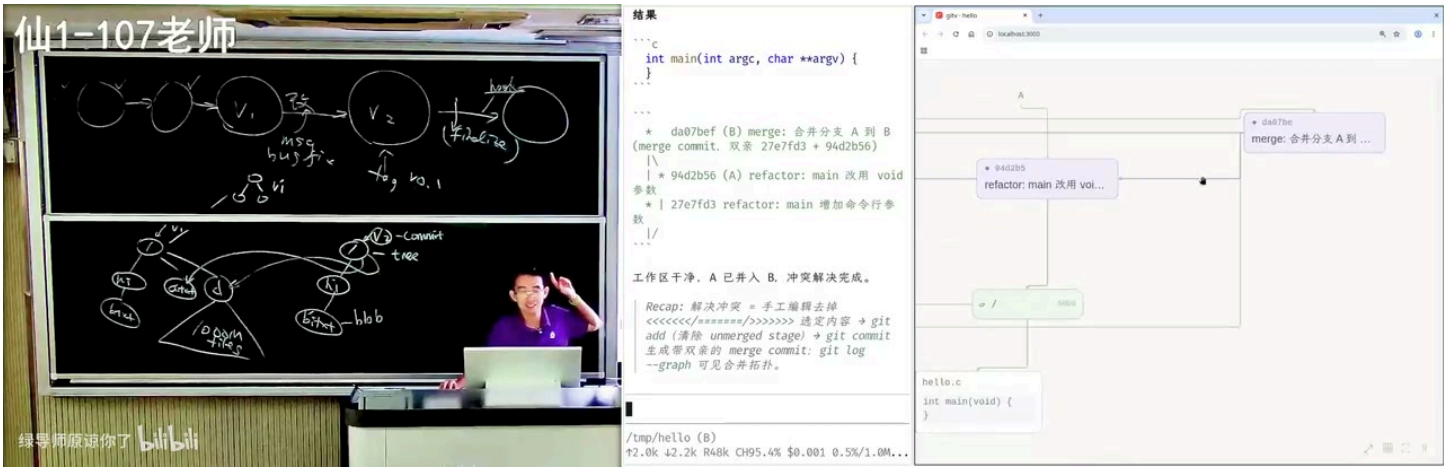
在 B 上合并 A 时，工作区 hello.c 出现双方文本与冲突标记。Index 同时保留 **base**、**ours**、**theirs** 三个版本，图里显示三张相应卡片；这比只知道“合并失败”更具体地说明了待解决的信息。




**视频原帧** 冲突现场：工作区文本标记与 Index 的 base/ours/theirs 并排呈现。

视频原帧 01:17:20 · 冲突现场：工作区文本标记与 Index 的 base/ours/theirs 并排呈现。

解决冲突后，未合并的临时状态消失，形成带**两个 parent** 的 merge commit。现场结果保留 argc/argv 形式；历史图同时记住两条来路，因此合并不是简单把某一个旧版本改个名字。




**视频原帧** 冲突解决后：合并提交保留两个父提交，最终代码采用 argc/argv 形式。

视频原帧 01:17:58 · 冲突解决后：合并提交保留两个父提交，最终代码采用 argc/argv 形式。

---

第 20 节

01:18:09—01:22:14 · 课堂时间

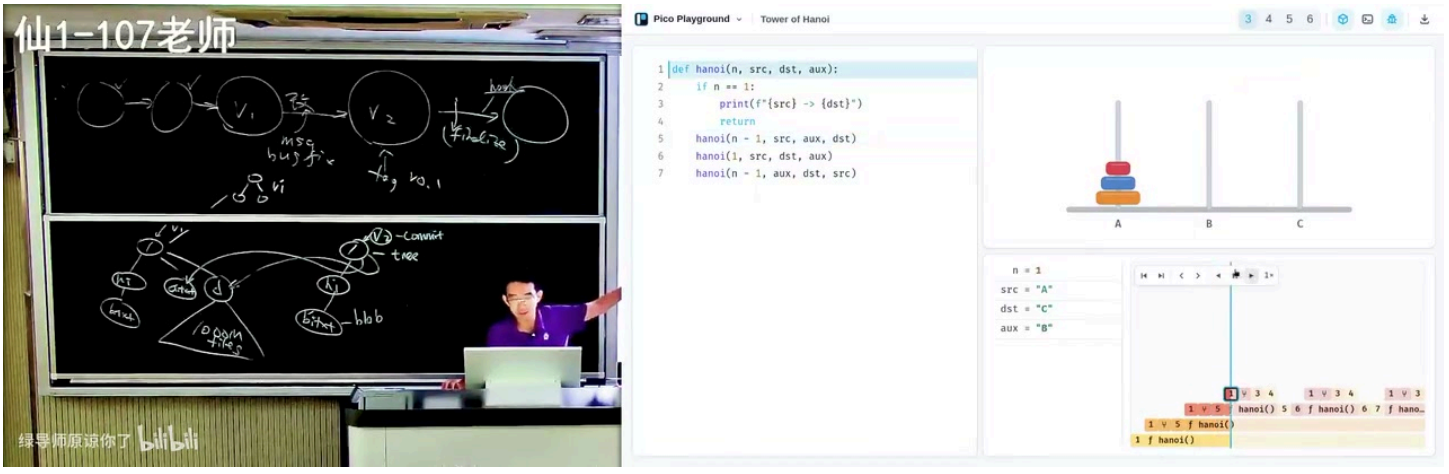
# Pico Playground：先让孩子觉得编程可玩

> **关键判断 · 整理者提炼**
>
> 可视化应让学习者在统一时间点观察代码、变量与动作，并预测执行的下一步。

讲师打开 code.cs61a.org 的 Python IDE，与自己参与设计的 Pico Playground 作比较。课堂访问前者时有加载、登录限制，因此没有完成等条件功能对照；本段是设计案例，不是正式产品评测。

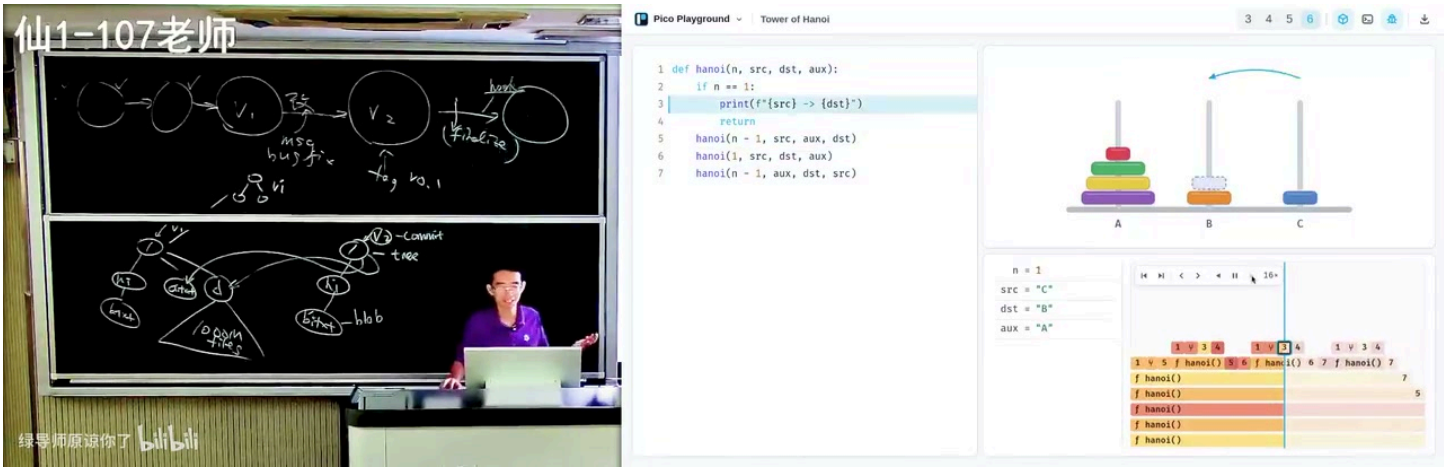
Pico 的目标面向小学三四年级孩子：第一课不应被完整专业 IDE 压住，而应通过**极简界面**感受到“Python 很好玩”。演示包括整数与 Unicode 字符互转、hello 乘以 10 生成重复字符串。

汉诺塔例子把递归代码、当前变量、**调用/执行轨迹**、盘子的移动同步显示。讲师可以前后拖动时间轴、逐步执行或播放，让学生在动作发生前**预测**接下来会怎样。




**视频原帧** Pico 汉诺塔：代码、变量、调用轨迹、移动画面在同一时间点联动。

视频原帧 01:21:46 · Pico 汉诺塔：代码、变量、调用轨迹、移动画面在同一时间点联动。




**视频原帧** Pico 汉诺塔后续阶段：增加盘数后继续播放，盘子分布、移动箭头与当前变量同步改变。

视频原帧 01:21:59 · Pico 汉诺塔后续阶段：增加盘数后继续播放，盘子分布、移动箭头与当前变量同步改变。

该系统可做网页应用，也可包装为桌面应用；讲师称自己主要把控架构，具体展示由 AI 生成。这解释了为什么程序、轨迹与多种可视化必须有**共同接口**，而不是每个演示各写一套相互缠绕的状态。

---

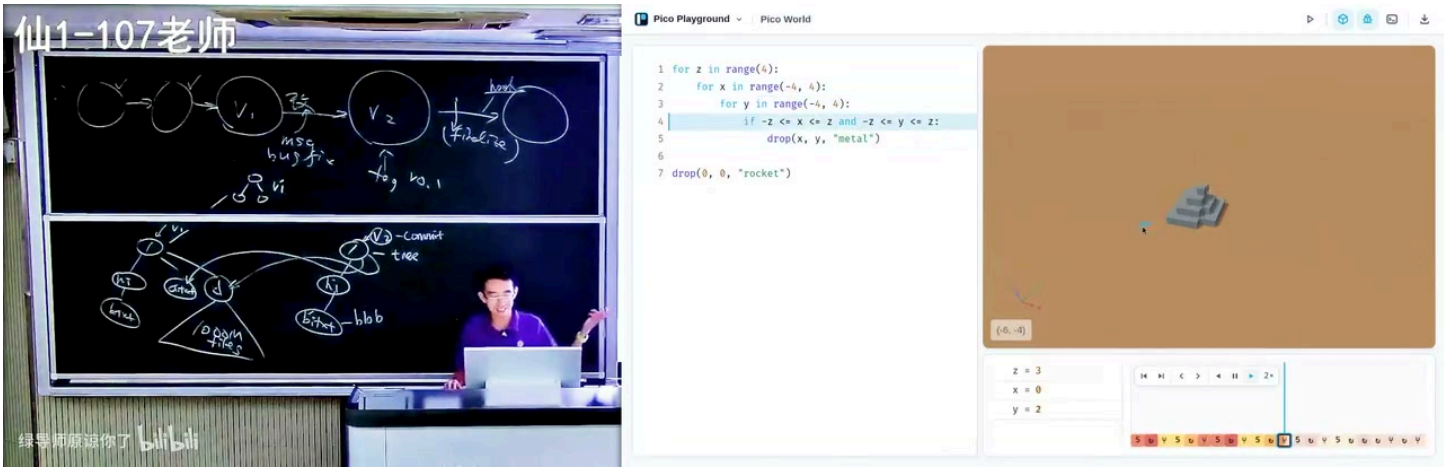
第 21 节

01:22:14—01:26:26 · 课堂时间

# 把循环变成立体世界：细节与学习标准

> **关键判断 · 整理者提炼**
>
> 教学可视化需要针对学习目标反复验收，程序能运行只是其中一部分。

另一个演示为孩子提供简单 API 和可操作的世界。几重循环调用 drop，把方块堆成空间结构；运行过程可以向前、向后拖动，每一步与源代码、坐标和变量相对应。

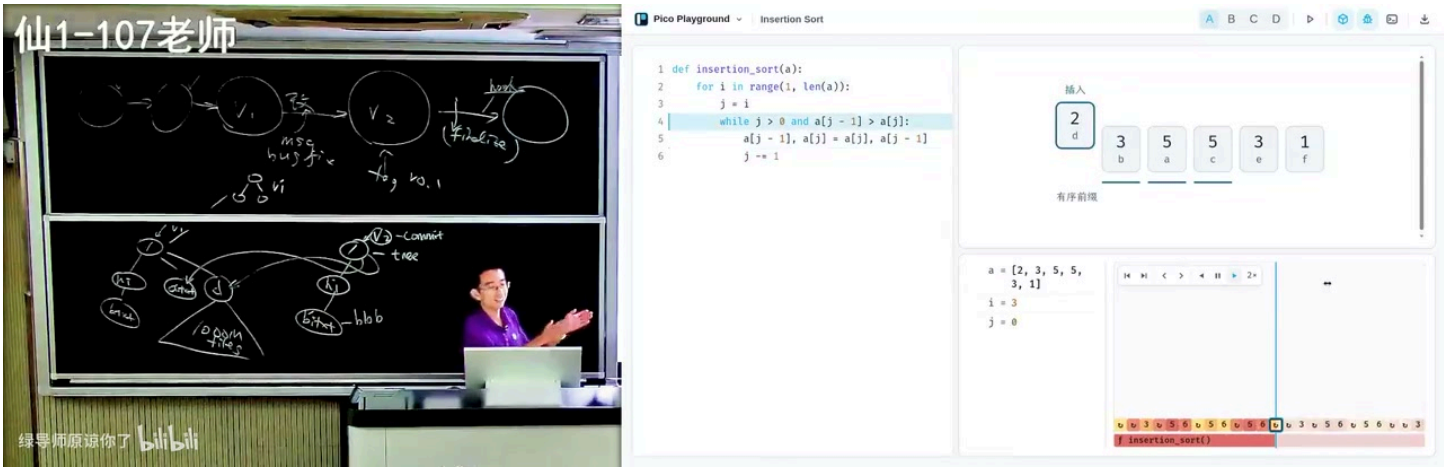



**视频原帧** Pico World：循环、drop API、坐标、状态与方块世界同步。

视频原帧 01:23:05 · Pico World：循环、drop API、坐标、状态与方块世界同步。

讲师说第一版坐标轴不符合预期，反馈后才达到他想要的表示，动画也经过调整。这里说明人的验收仍在：不只程序运行，还要检查呈现是否符合**教学目标**和**空间直觉**。

课堂随后切到**插入排序**：当前插入元素、已排序前缀、比较交换、变量与轨迹一起出现。讲师用一个 skill 把想教的程序编译成此类课件，并称便宜模型也能达到此次任务的可用程度。




**视频原帧** 插入排序：当前插入项、已排序前缀、变量和执行轨迹。

视频原帧 01:24:08 · 插入排序：当前插入项、已排序前缀、变量和执行轨迹。

这段回到学习意识：愿意亲自打开优秀作品、追问每个细节的人，能够不断提高自己的标准。两天碎片时间完成框架是讲师的个人经历，不是对所有人或所有项目的工时承诺。

# 愿意学懂，成了真正有价值的品质

## “把细节做好” 的 Bar

- 大学里没训练过，真上了战场就都得死
- 很多大学老师真正的好东西打开都没打开过，就只会照本宣科、断章取义
  - 我真的无法接受看公众号学习依然心安理得的老登

## 好消息：真正学懂和成绩是正相关的

- **愿意学懂**，就成了真正有价值的品质。

## 遗憾的是，自己的学习时间非常有限






**资料图 · 第 23 页** 学习工具降低操作成本后，愿意学懂与留意细节更有价值。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 23 · 原始讲义 · 学习工具降低操作成本后，愿意学懂与留意细节更有价值。

---

第 22 节

01:26:26—01:31:27 · 课堂时间

# 提交说明、及时检查点与协作约定

> **关键判断 · 整理者提炼**
>
> 及时保存探索检查点，再整理成可解释的变化，才能支持恢复、审查与协作。

回到 Git，讲师注意到 AI 前面写英文提交，后面又混入中文。问题不只在语言选择，而是一个项目中约定不一致，迫使维护者不停切换理解方式。文档中中英文间空格的例子也服务于这一点，并非给所有场景规定唯一排版。

Conventional Commits 用类型、可选范围和行为变化组织标题；**正文解释**触发条件、修改动机和保留的行为。课堂展示了标题与正文的区别，讲义例子 fix(parser): accept repeated spaces between arguments 把范围和结果说清楚。

# 更多的 Best Practices

## 比如，你会写 Commit Message 吗?

- Conventional Commits
- ✦ `fix(parser): accept repeated spaces between arguments`
  - ✦ 类型 + 范围 + 行为变化：人可以浏览，工具可以分类、生成 changelog。

## Diff 留下改动，Message 留下理由

- ✦ 正文示例：旧解析器把连续空格当成空参数，导致复制来的命令失败；现在忽略多余空格，保留引号内空格。
- ✦ `fix`：只能说明“修了 bug”；触发条件、设计理由和 issue 链接，才能帮助后来的人判断“这段代码能不能删”。



**资料图 · 第 24 页** Diff 留改动，提交说明留理由；格式只是入口。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 24 · 原始讲义 · Diff 留改动，提交说明留理由；格式只是入口。

及时 commit 给探索留**检查点**，发布前再整理成围绕单个意图的可理解变化。一个巨大提交若引入很多 bug，之后所有修复都指向它，就难以判断哪条需求已经修好；较小且完整的提交更利于 review、revert、bisect 和追踪。

# Best Practices：让错误可恢复

## 一、给自己留退路

还有随处可见的 best practice。

- ✦ **及时 commit**：实验前留检查点；只在工作区里的修改，Git 未必能救。
- ✦ **备份与恢复**：误操作后先保护现场，再查 reflog；远端 clone 不包含未提交文件。

## 二、把探索整理成别人能接手的变化

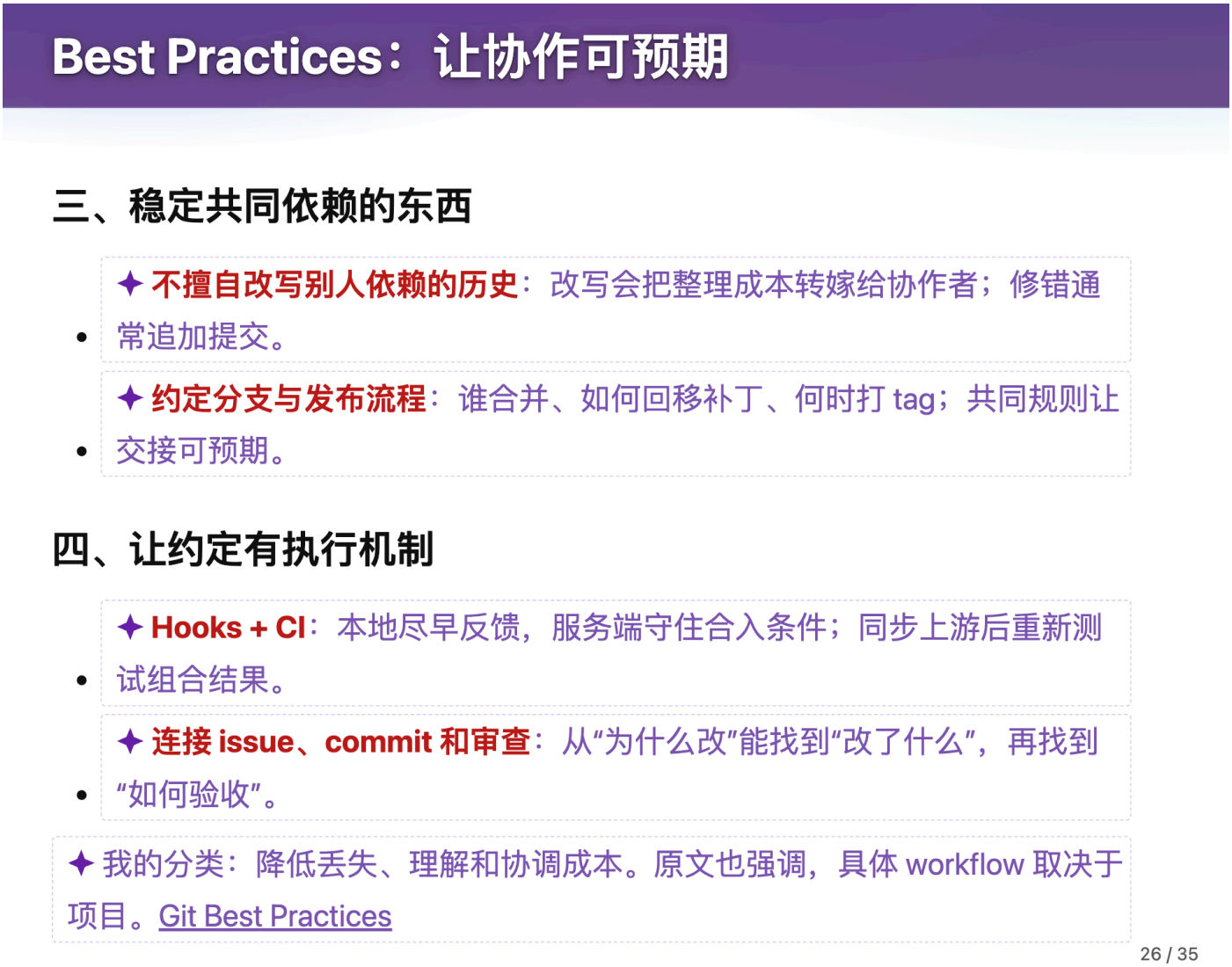
- ✦ **发布前整理**：按一个意图拆分提交，把修复和相应测试放在一起；顺手格式化另开提交。
- ✦ **让历史可检验**：整理后的提交能构建、能测试，review、revert、bisect 才有可靠的单位。



**资料图 · 第 25 页** 留退路与整理变更：检查点、恢复和可验证历史。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 25 · 原始讲义 · 留退路与整理变更：检查点、恢复和可验证历史。

远端备份需要推送已保存的历史；本地未提交的编辑不会因为有远端仓库就自动被保存。Hooks 和 CI 把部分约定落实为检查，但自己的修改通过后，与新主线**组合**仍可能需要再测。




**资料图 · 第 26 页** 协作约定：共享历史、流程、Hooks/CI 与追溯。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 26 · 原始讲义 · 协作约定：共享历史、流程、Hooks/CI 与追溯。

> **整理者说明**
> 讲义扩展将实践分成降低丢失、理解和协调成本；具体 workflow 依项目决定，不是唯一标准。

---

第 23 节

01:31:28—01:33:27 · 课堂时间

# Git 之外还有平台：gh/glab 与完整工程任务

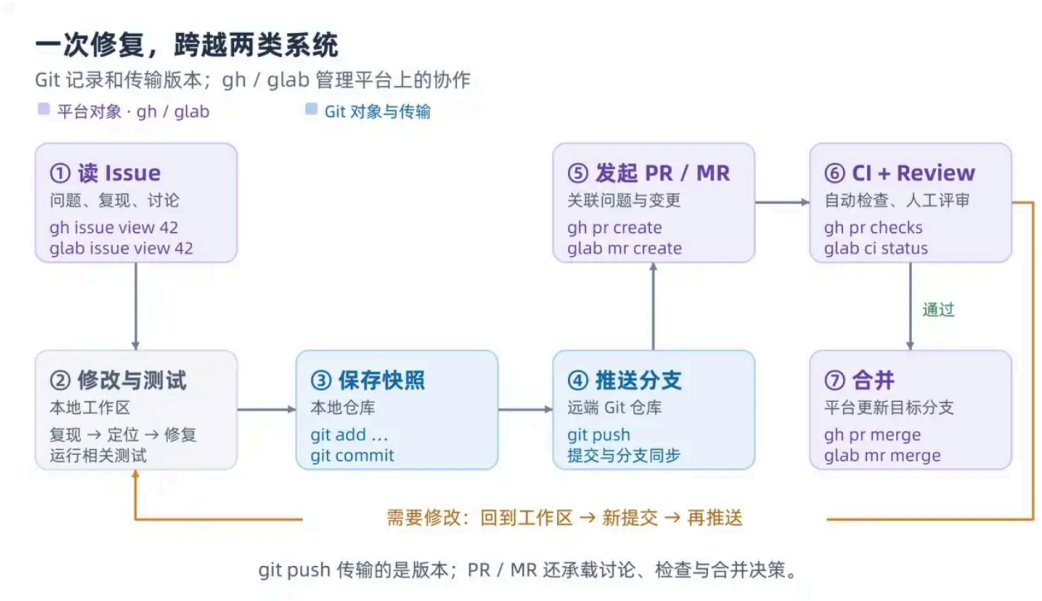
> **关键判断 · 整理者提炼**
> Git 保存版本，平台保存问题与审查状态，工程交付要把改变理由、实现和验证连起来。

Git 管理项目版本；**GitHub/GitLab** 上的 issue、评论、审查和 CI 状态不是随 Git tree 一起保存的普通版本对象。**gh**、**glab** 把这些平台能力接进命令行，便于人和 Agent 读取、处理并连接工作。

# Git 之外：gh 与 glab

## 从问题到合入，把流程接起来

甚至为了管理 non-git 的部分，还专门有 gh, glab 这些工具。




✦ Git 保存和传输版本；gh / glab 操作平台上的 issue、PR/MR 与 CI 状态，让人和 Agent 都能从命令行完成协作。

29 / 35

**资料图 · 第 29 页** gh/glab 连接平台对象；Git 保存与传输版本。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 29 · 原始讲义 · gh/glab 连接平台对象；Git 保存与传输版本。

课堂描述的工作流包括找分配给自己的 issue、完成修改与小步提交、按协作约定推进，再更新平台状态。讲师展示自己项目中的连续提交，强调长期任务也应留下可解释的步骤。

这里的思想是将“**为什么改**”“改了什么”“**怎样验证**”连起来。版本工具和平台工具各自承担一部分，不宜把 git push 当成已经完成沟通、审查和交付。

> **整理者说明**
>
> 讲义补充：SWE-bench 要求 **FAIL_TO_PASS** 从失败变通过、PASS_TO_PASS 保持通过。Astropy astropy__astropy-12907 案例中，右侧子模型依赖矩阵被全1覆盖，修复为保留 right；真实 PR 同时补测试与 changelog，并回移维护分支。这是现行讲义材料，课堂转写未展开，亦未在本次运行中独立执行该 PR。

# SWE-Bench：接手已有项目的一次修改

## 输入是项目现场，输出是补丁

然后你会发现，这些任务和软件工程日常任务是关联非常大的。

- ✦ 经典 SWE-bench 从真实 GitHub issue 和对应 PR 构建任务：给定仓库、起始提交 `base_commit` 和问题描述，产出 patch。任务定义
- ✦ 日常工作顺序：理解现象 → 复现 → 找调用路径和已有约定 → 修改 → 验证 → 检查 diff。

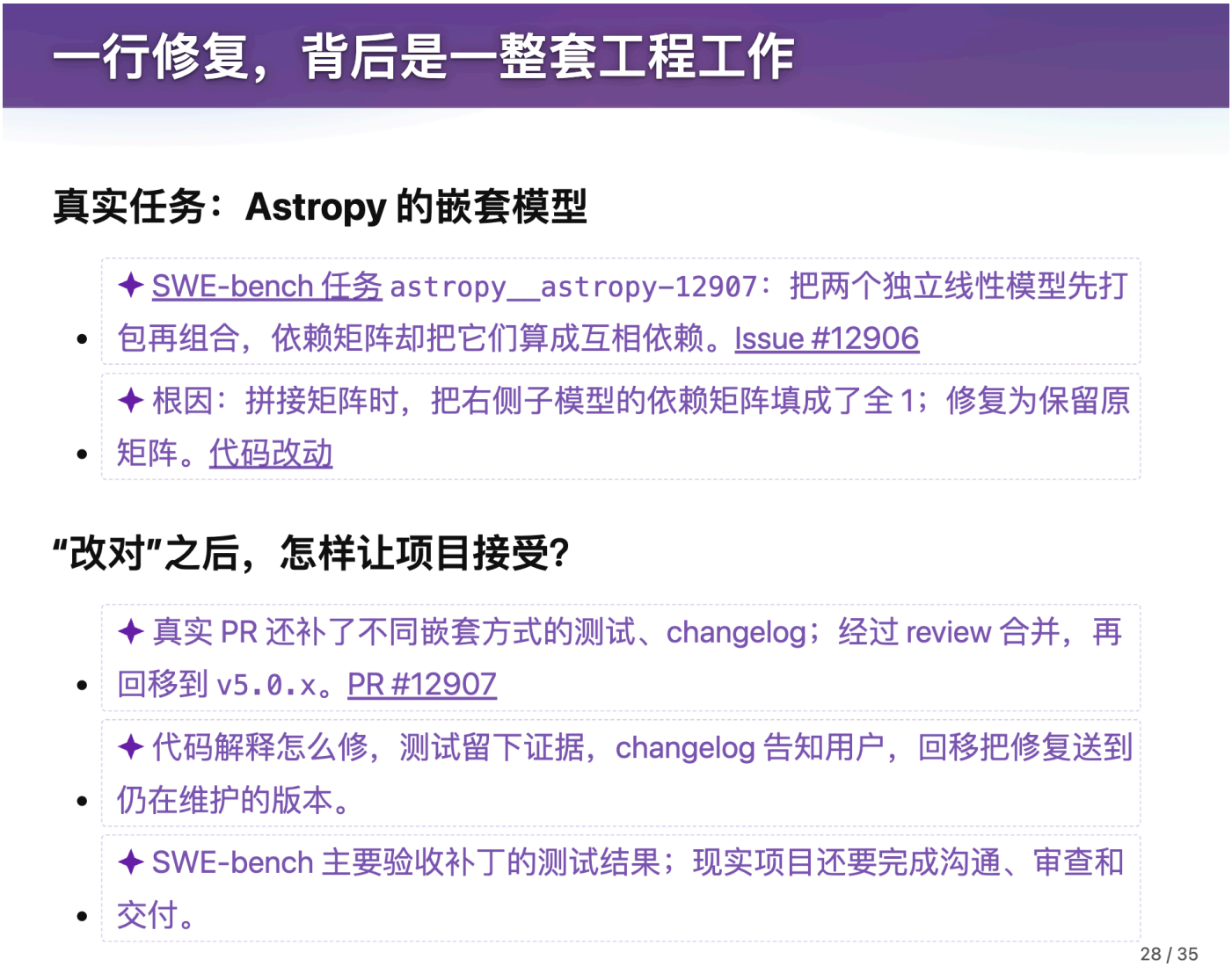
## 修好这个问题，同时守住已有行为

- ✦ `FAIL_TO_PASS`：针对问题的测试，由失败变为通过。
- ✦ `PASS_TO_PASS`：原来通过的测试，修改后仍然通过。评测规则
- ✦ 小而完整的改动便于定位失败；固定起点让别人能重放修改。前面的习惯，在这里都有用。



**资料图 · 第 27 页** 讲义补充 | SWE-bench：给定仓库起点与问题，产出修复补丁；课堂未展开此页。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 27 · 原始讲义 · 讲义补充 | SWE-bench：给定仓库起点与问题，产出修复补丁；课堂未展开此页。




**资料图 · 第 28 页** 讲义补充 | Astropy 嵌套模型：一行修复之外还有测试、更新记录与发布。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 28 · 原始讲义 · 讲义补充 | Astropy 嵌套模型：一行修复之外还有测试、更新记录与发布。

---

第 24 节

01:33:30—01:36:36 · 课堂时间

# 项目由原子变更推进：可解释、验证和撤销

> **关键判断 · 整理者提炼**
>
> 原子变更围绕一个完整意图组织代码、测试和文档，使每一步可解释、验证与撤销。

本课的核心归纳是：项目是由 atomic change 推进的快照，可以分支，也可以合并。人的理解能力有限，难以审查一次混合很多需求、很多文件和多种目的的“一团混沌”。




## 项目 = 由 Atomic Change 推进的快照

可以分支，可以合并。这是人类成功的经验。

- 人无法处理“一团混沌”
- 必须分解成可以理解、可以审计的小任务
  - 出了问题可以 bisect，发生冲突可以解决
  - 有些项目要求 linear history：历史易于追溯，cherry-pick / blame 友好

## AI-native 也要避免“一团混沌”

- 在无限大能力模型到来之前，这条法则依然成立
- ✦ **一个 change 对应一个意图**：出了问题，才能单独解释、验证、撤销。

30 / 35

**资料图 · 第 30 页** 原子变更推进快照：减少理解和审计“一团混沌”的困难。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 30 · 原始讲义 · 原子变更推进快照：减少理解和审计“一团混沌”的困难。

因此把需求拆成可理解、**可审计**的小任务。探索失败可以舍弃该分支的尝试，成功再合并；小变更的冲突更容易定位，未冲突的独立部分也更容易协调。

原子不是“只改一行”或“只改一个文件”，而是围绕一个意图的**完整变化**。相应代码、测试和必要文档可以共同构成一个单位，便于单独解释、验证和撤销。

有些项目采用**线性历史**，使追踪、cherry-pick 和 blame 更方便。讲师也承认这不是全是优点；线性是组织选择，不能替代每一步的语义和可验证性。在模型仍有能力边界时，AI 参与的开发同样受益于这种结构。

---

第 25 节

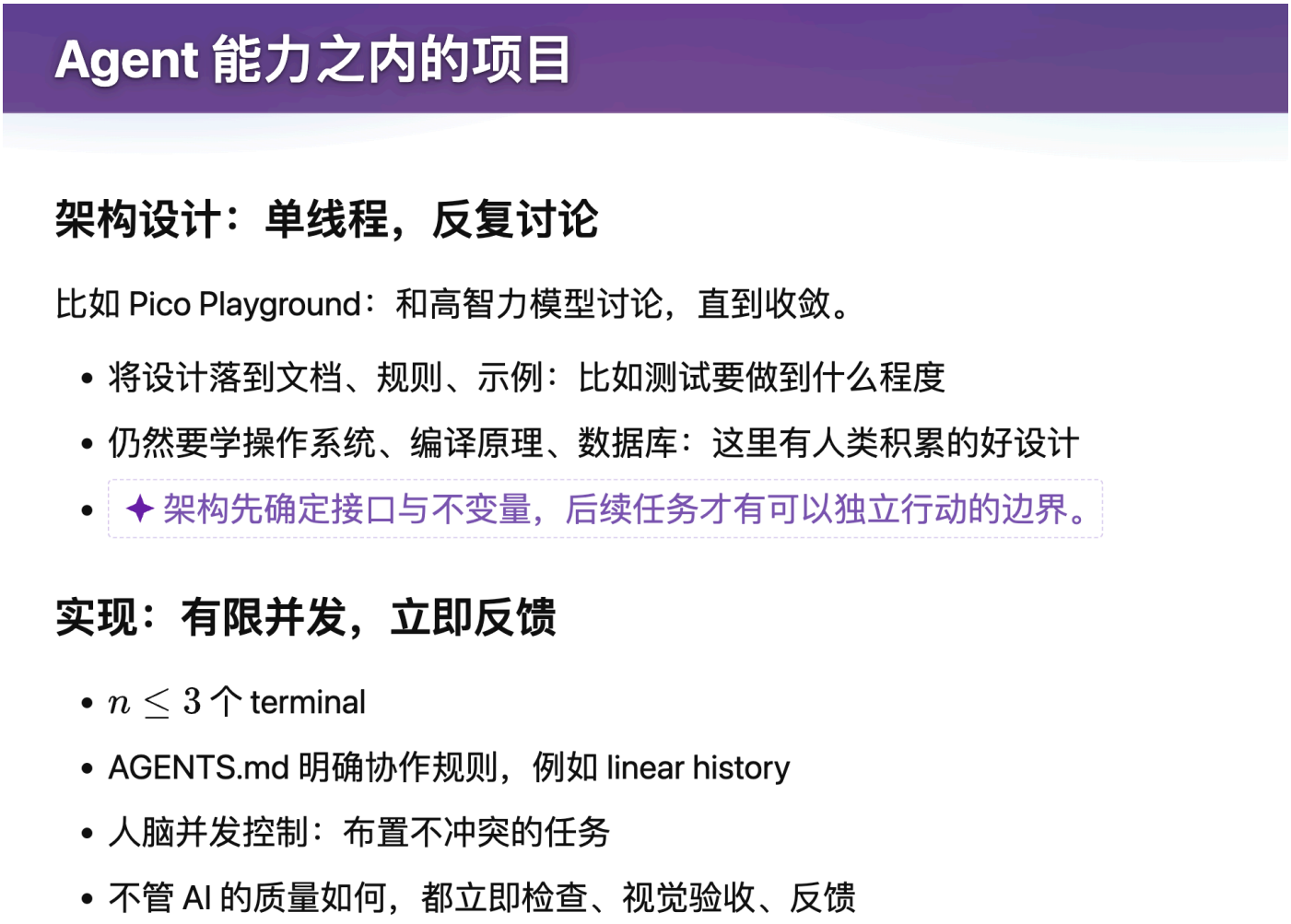
01:36:36—01:40:12 · 课堂时间

# 架构先收敛，再有限并发；未来 VCS 留待下课

> 关键判断 · 整理者提炼

> 先明确架构与接口再并行，而更快 Agent 的版本准入仍需要验证组合结果和恢复能力。

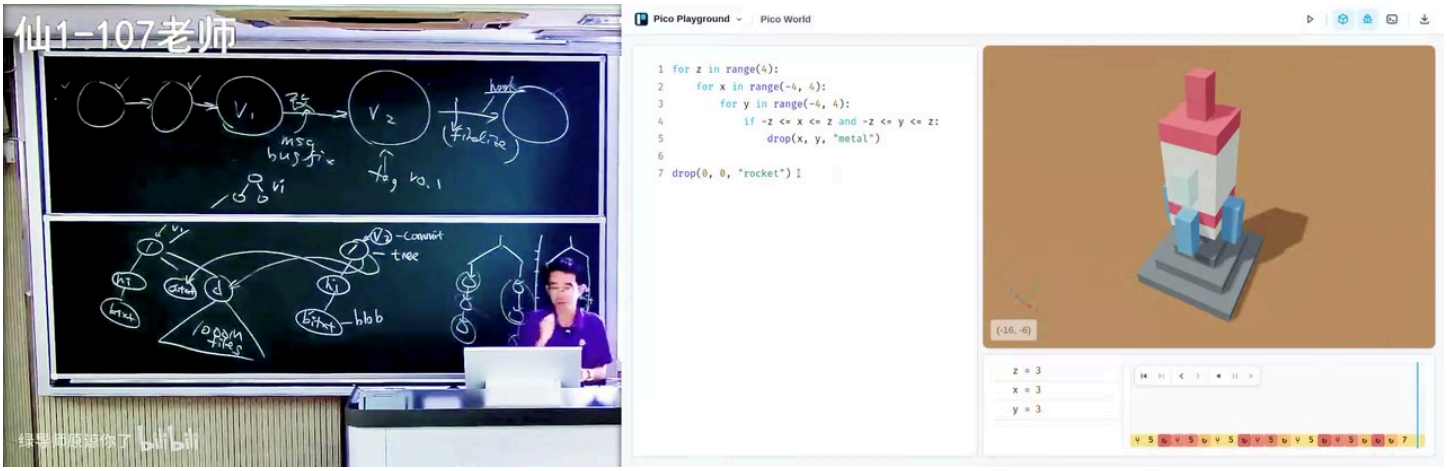
讲师的项目做法先是“单线程”架构讨论：与较强模型反复确认接口、约束、测试标准，再把结论落到文档和规则中。前提不清楚就急着并行，后面会不断返工。




**资料图 · 第 31 页** 架构讨论先收敛，再有限并发、立即反馈。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 31 · 原始讲义 · 架构讨论先收敛，再有限并发、立即反馈。

Pico 的具体例子是尽量用**纯函数**组织接口：程序产生**执行 trace**，展示由 trace 等输入推导；光标、菜单等确需互相影响的状态集中在小而明确的共享区域。讲师称核心共享状态与组装代码规模很小，其余模块作为黑盒使用。




**视频原帧** Pico 架构说明现场：代码、轨迹与世界联动；口述强调纯函数和小型共享状态。

视频原帧 01:38:10 · Pico 架构说明现场：代码、轨迹与世界联动；口述强调纯函数和小型共享状态。

他甚至要求生成可视化时只读库的接口，不读内部实现，以限制上下文和耦合。这里依赖**接口清楚**、模块职责明确；“只看接口”不是对所有调试任务的通用禁令。

架构稳定后，开少量终端推进**互不冲突的任务**，以 AGENTS.md 约定协作，由人脑承担一部分并发控制。讲师的习惯是不超过约三个终端，旁边浏览器即时刷新，自己做视觉反馈；这是个人方式，不是 Agent 的普遍并发上限。

最后仅提出思想实验：如果 Agent 再快一千倍，版本控制和人的验收怎样跟上？视频到此下课，明确说下一节再讨论。现行讲义32—35页已提供后续设计方向，本节作为补充保留，不写成这节实际完成的口述推导。

---

官方现行讲义，视频结尾仅提出问题并留待下一课

## 课堂未展开：更快Agent的版本控制设想

讲义设想让Agent在独立快照生成候选变更，系统读取当前版本H，将变化组合成C，在锁外完成测试与**不变量**验证；发布时短锁或CAS确认HEAD仍为H，只原子发布已经验证的**确切C**。若主线已变，需重新读取、组合和验证，不能把对旧候选的通过套到新组合。MVCC处理多版本并发访问，change algebra描述变更组合，二者相关但不相同。可交换公式ΔB(ΔA(S))=ΔA(ΔB(S))也不自动保证业务正确。

---

PDF物理页 32、33 · 原始资料 ↗

# 当 Agent 快了 1000 倍

## 一个思想实验：代码产能暴涨

- Agent 可以 1000X 更快地生产代码
- Git“为人设计”的使用方式，可能跟不上这个速度
  - ✦ 生成一分钟，人工验收一小时：积压的是等待判断的变更。
  - ✦ 即使修改不同文件，也可能同时改变同一个接口约定。

## 从零设计一个 Agent VCS

- 快照、变更，以及一个 change algebra
- ✦ 系统要回答：哪些变更能组合、哪些必须排队、哪些需要重新验证?
- ✦ 像数据库处理事务一样，让 Agent 提交“基于某个版本的修改”，由系统控制它何时生效。

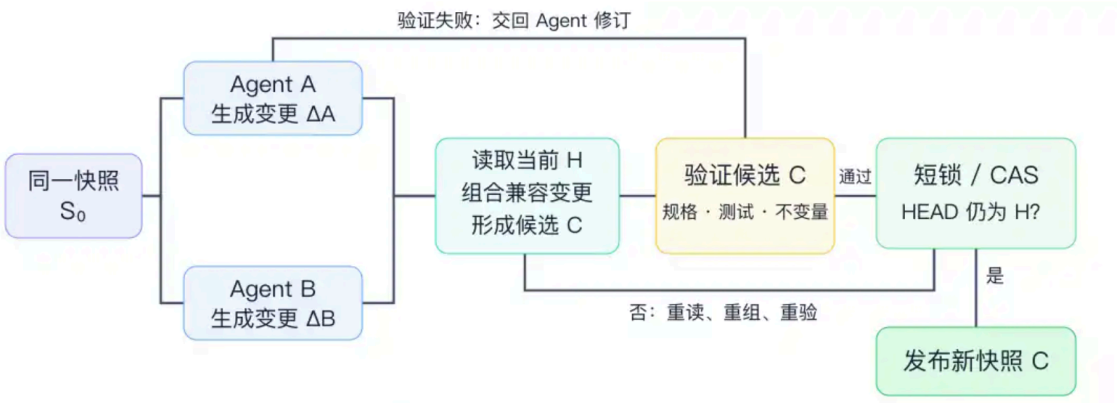


**资料图 · 第 32 页** 讲义补充｜1000倍是放大瓶颈的思想实验，课堂只提出问题。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 32 · 原始讲义 · 讲义补充｜1000倍是放大瓶颈的思想实验，课堂只提出问题。

# 快照、变更与并发控制

## 在快照上并行工作，在发布时决定顺序




MVCC 管理并发版本；Change Algebra 描述变更如何组合

✦ **MVCC** 管理多个版本；**change algebra** 描述变更怎样组合。可交换的变更满足 $\Delta_B(\Delta_A(S)) = \Delta_A(\Delta_B(S))$，但组合结果仍需满足项目的不变量。

✦ 图为一种设计设想：生成与测试在锁外进行；只对验证过的确切版本做原子发布。版本已变，就重新组合、验证。参考：PostgreSQL 并发控制。

33 / 35

**资料图 · 第 33 页** 讲义补充｜独立快照生成、候选组合验证、确切版本原子发布。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 33 · 原始讲义 · 讲义补充｜独立快照生成、候选组合验证、确切版本原子发布。

官方现行讲义补充，不是本课已完成的现场演示

## 课堂未展开：无文本冲突的write skew与审计边界

讲义用至少一名值班员在岗为**不变量**：甲乙都在岗时，两个Agent分别看到对方在岗，便把自己负责的一人设成离岗。两个文件合并无冲突，最后却无人值班。需要组合结果验证或更强并发控制。Jujutsu的稳定change ID、可保存冲突、operation log是相关机制，不等于已经实现课堂设想的自动准入系统。人的大步历史之外，机器可保留细粒度操作及验证证据；“修好就行”仅在错误能及时发现、后果可恢复时才可能成立。测试全绿只覆盖现有检查，审计仍要问系统凭什么接受变化、漏了什么约束。

PDF物理页 34、35 · 原始资料 ↗

# 合并成功之后，还要问什么？

## Jujutsu 已经靠近这个方向

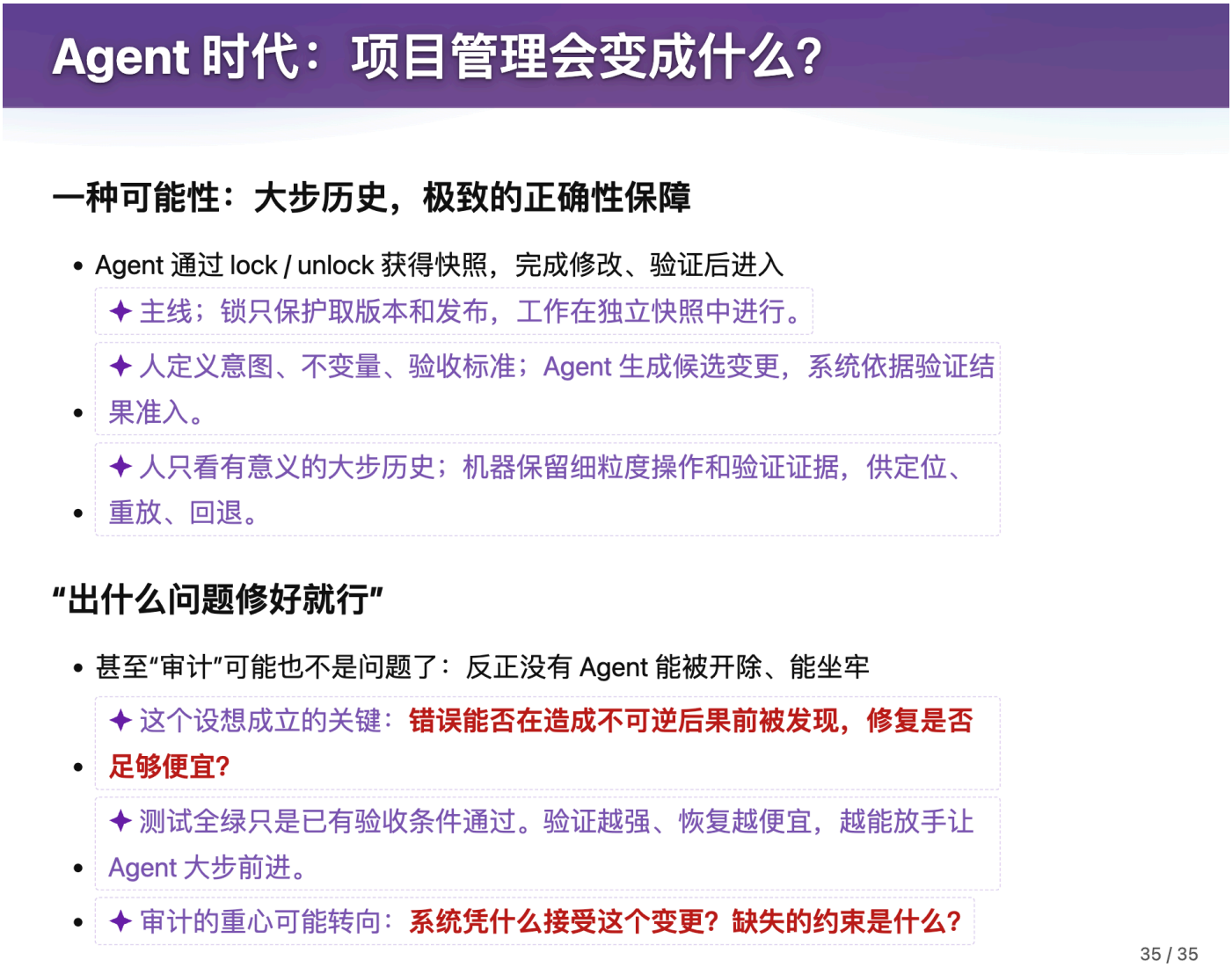
- ✦ 稳定的 change ID：一个变更经过修改、rebase，仍能追踪其身份。见 Jujutsu Tutorial。
- ✦ 冲突可以作为状态保存；operation log 记录仓库操作，并支持并发操作的协调。见 First-Class Conflicts 与 Operation Log。

## 没有文本冲突，也可能违反约定

- ✦ 项目要求：至少一名值班员在岗。快照中，甲、乙都在岗。
  - ✦ Agent A 看到乙在岗，把甲改成离岗；Agent B 看到甲在岗，把乙改成离岗。
  - ✦ 即使修改分别落在两个文件里，合并后也会无人值班。
- ✦ 数据库里，这叫 **write skew**：只提供快照还不够，需要约束验证或更强的并发控制。参考：事务隔离。



**资料图 · 第 34 页** 讲义补充 | Jujutsu 与 write skew：没有文本冲突也可能违反共同约束。




**资料图 · 第 35 页** 讲义补充 | 机器细粒度证据与人类大步历史；错误恢复能力是前提。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 35 · 原始讲义 · 讲义补充 | 机器细粒度证据与人类大步历史；错误恢复能力是前提。

---

## 自测：先回答，再核对

先用自己的话回答，再展开参考答案；答不清楚时，回到对应知识点。

问题 01

**add 之后再修改文件，普通 commit 提交哪个版本?**

参考答案 · 核对思路

**暂存区**保存的版本；后续编辑仍在工作区，需再次 add 才进入该提交。

回查第 10 节 →

问题 02

### 为什么逻辑上的完整快照不意味着复制所有文件?

参考答案 · 核对思路

变化内容与目录路径可以生成新对象，未变部分**共享**；逻辑状态与存储复用是不同层次。

回查第 16 节 →

问题 03

### 解决文本冲突时，工作区与 Index 各提供什么?

参考答案 · 核对思路

工作区展示带标记的待决文本，Index保留base/ours/theirs等信息；人或Agent依据需求决定最终内容，再完成合并。

回查第 19 节 →

问题 04

### 什么样的修改算原子变更?

参考答案 · 核对思路

围绕一个明确意图，能单独解释、验证和撤销的完整修改，可能跨代码、测试和文档。

回查第 24 节 →

问题 05

### 两个Agent改不同文件，为何仍可能合出错误?

参考答案 · 核对思路

它们可能依据同一旧快照分别放松条件，组合后违反共同**不变量**；文本无冲突并不证明业务约束成立。

回查第 25 节 →